

INFORMATIQUE • TLE C

MODULE 3 : ALGORITHMIQUE ET PROGRAMMATION

LEÇON N07

ÉCRITURE ET EXÉCUTION D'ALGORITHMES INTÉGRANT UNE STRUCTURE DE DONNÉES

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon introduit les structures de données (variables, constantes, enregistrements, tableaux) et montre comment les intégrer dans des algorithmes complets. Elle culmine avec l'écriture et l'exécution pas à pas de deux algorithmes classiques du programme : la recherche séquentielle et le tri par insertion.

- À la fin de cette leçon, je sais décrire la structure générale d'un algorithme (entête, déclarations, corps) et déclarer variables et constantes.
- Je sais distinguer les structures de données finies, indexées et récursives, et citer un exemple de chacune.
- Je sais déclarer un tableau, le remplir, le parcourir et accéder à l'élément d'indice i .
- Je sais déclarer un enregistrement et accéder à ses champs.
- Je sais écrire l'algorithme de recherche séquentielle dans un tableau à l'aide d'un témoin (drapeau).
- Je sais écrire et exécuter pas à pas l'algorithme de tri par insertion sur un tableau d'entiers.



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Rappels : structure d'un algorithme et instructions de base
2. Les structures de données
3. Tableaux et enregistrements
4. Recherche séquentielle dans un tableau
5. Tri par insertion
6. Exécution pas à pas et table d'exécution
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais décrire la structure générale d'un algorithme (en-tête, déclarations, corps) et déclarer variables et constantes.
- Je sais distinguer les structures de données finies, indexées et récursives, et citer un exemple de chacune.
- Je sais déclarer un tableau, le remplir, le parcourir et accéder à l'élément d'indice i .
- Je sais déclarer un enregistrement et accéder à ses champs.
- Je sais écrire l'algorithme de recherche séquentielle dans un tableau à l'aide d'un témoin (drapeau).
- Je sais écrire et exécuter pas à pas l'algorithme de tri par insertion sur un tableau d'entiers.
- Je sais construire une table d'exécution (lignes, instructions, variables, écran) pour vérifier un algorithme.
- Je sais résoudre un problème concret de recherche ou de tri de données dans un tableau.

PRÉREQUIS

- Notion d'algorithme et d'organigramme (Première)
- Instructions simples : affectation, lecture, écriture
- Structures de contrôle alternatives (Si...Alors...Sinon) et itératives (Pour, TantQue)

- Types de données de base : entier, réel, caractère, chaîne, booléen

1 RAPPELS : STRUCTURE D'UN ALGORITHME ET INSTRUCTIONS DE BASE

Le proviseur du lycée de Bafoussam dispose d'une liste de 500 élèves avec leurs moyennes, saisies dans le désordre. Il veut savoir si un élève précis figure dans la liste, puis classer les élèves par moyenne décroissante pour le tableau d'honneur. Faire cela à la main prendrait des heures et serait source d'erreurs : il faudrait parcourir 500 lignes une à une, comparer, recopier, sans jamais se tromper. Un ordinateur, lui, fait cela en une fraction de seconde, à condition de lui fournir une recette sans ambiguïté : un **algorithme**. Comment organiser les données en mémoire et écrire un algorithme capable de rechercher et de trier automatiquement ces informations ? C'est tout l'objet de cette leçon. Mais avant de manipuler des tableaux de 500 élèves, il faut maîtriser les fondations : la structure d'un algorithme et ses instructions de base. C'est l'objet de cette première section.

1.1 Qu'est-ce qu'un algorithme ?

DÉFINITION Algorithme

Un **algorithme** est une suite finie et ordonnée d'instructions précises et non ambiguës, permettant de résoudre un problème ou d'obtenir un résultat à partir de données fournies en entrée.

L'intuition est simple : un algorithme est une recette de cuisine. La recette du ndolé indique les ingrédients (les données), les étapes dans l'ordre (les instructions) et le plat obtenu (le résultat). Comme une recette, un algorithme possède des propriétés essentielles :

- il est **fini** : il se termine après un nombre fini d'étapes ;
- il est **précis** : chaque instruction a une seule interprétation possible ;
- il est **effectif** : chaque instruction peut réellement être exécutée ;
- il produit un **résultat** à partir de données d'entrée.

LESAVAIS-TU ? D'où vient le mot « algorithme » ?

Le mot vient du nom du mathématicien perse Al-Khawarizmi (IX^e siècle), dont les ouvrages décrivaient des méthodes de calcul systématiques. Au Cameroun, les algorithmes sont partout : la répartition automatique des élèves par le système OBC (Orientation Baccalauréat Concours), le calcul des soldes Mobile Money, ou encore le classement des candidats aux concours d'entrée dans les grandes écoles.

1.2 Structure générale d'un algorithme

Tout algorithme écrit en pseudo-code suit une architecture en trois parties, toujours dans le

même ordre :

1. **L'en-tête** : le mot-clé `Algorithme` suivi du nom de l'algorithme ;
2. **La partie déclarative** : on y annonce toutes les constantes et toutes les variables utilisées, avec leurs types ;
3. **Le corps** : délimité par les mots-clés `Début` et `Fin`, il contient les instructions exécutées dans l'ordre.

En-tête : Algorithme TableauHonneur

Partie déclarative

Constante N = 500
Variables i, somme : Entier
Variables moyenne : Réel

on annonce les objets

on exécute les actions

Corps de l'algorithme

Début
instructions (lecture, calculs, affichage)
Fin

Structure générale d'un algorithme : en-tête, partie déclarative, corps.

ATTENTION Erreur fréquente

Utiliser dans le corps une variable qui n'a pas été déclarée dans la partie déclarative est une erreur. Toute variable ou constante utilisée doit être annoncée avant le mot-clé `Début`. C'est comme inviter un élève à participer à un conseil de classe sans l'avoir inscrit sur la liste : le compilateur (ou le correcteur au Baccalauréat) le refusera.

1.3 Constantes et variables

Pour traiter des informations, l'algorithme les stocke en mémoire dans des « boîtes étiquetées ». Deux cas se présentent : soit la valeur ne change jamais (constante), soit elle peut évoluer (variable).

DÉFINITION Constante

Une **constante** est un objet mémoire dont la valeur est fixée une fois pour toutes dans la partie déclarative et ne peut pas être modifiée pendant l'exécution.

DÉFINITION Variable

Une **variable** est un objet mémoire identifié par un nom (l'identifiant) et doté d'un type, dont la valeur peut changer au cours de l'exécution grâce à l'affectation.

La syntaxe de déclaration est la suivante :

```
Constante TAUX = 18      // valeur fixe : TVA de 18 %
Constante N = 500        // nombre d'élèves du lycée
Variables i, compteur : Entier
Variables moyenne, somme : Reel
Variable nomEleve : Chaîne
Variable admis : Booleen
```

PROPRIÉTÉ Règles sur les identifiants et les types

Un identifiant commence par une lettre, ne contient ni espace ni caractère spécial, et ne doit pas être un mot réservé (Début, Si, Pour...). Les types usuels sont : Entier (ex. 25), Réel (ex. 13,75), Caractère (ex. 'A'), Chaîne (ex. "Ngo Bell") et Booléen (Vrai ou Faux). Le type détermine les valeurs possibles et les opérations autorisées : on peut additionner deux entiers, mais pas deux booléens.

1.4 Les instructions simples

Trois instructions suffisent pour faire entrer des données, les transformer et les restituer.

1. L'affectation. C'est l'instruction fondamentale : elle range une valeur dans une variable. On la note avec la flèche $\leftarrow$.

```
somme ← 0          // la variable somme reçoit la valeur 0
moyenne ← somme / N // on calcule puis on range le résultat
i ← i + 1           // la nouvelle valeur de i est l'ancienne + 1
```

PROPRIÉTÉ Sens de l'affectation

L'instruction $X \leftarrow \text{expression}$ s'exécute en deux temps : on évalue d'abord l'expression à droite (avec les valeurs actuelles des variables), puis on range le résultat dans la variable à gauche. L'ancienne valeur de la variable est définitivement perdue. Ainsi $i \leftarrow i + 1$ n'est pas une équation mathématique impossible, mais un incrément : si i valait 3, il vaut désormais 4.

2. La lecture (entrée). Lire(X) permet à l'utilisateur de saisir une valeur au clavier, qui est rangée dans la variable X .

3. L'écriture (sortie). Écrire(expression) affiche une valeur ou un message à l'écran.

EXEMPLE Un mini-algorithme complet

```

Algorithme ConversionFCFA
Constante EURO = 655,957 // 1 euro = 655,957 FCFA
Variables montantEuro, montantFCFA : Reel
Debut
    Ecrire("Montant en euros : ")
    Lire(montantEuro)
    montantFCFA ← montantEuro * EURO
    Ecrire("Equivalent en FCFA : ", montantFCFA)
Fin

```

Si l'utilisateur saisit 10, l'écran affiche : Equivalent en FCFA : 6559,57.

1.5 Rappel des structures alternatives

Un algorithme ne se contente pas d'exécuter des instructions en ligne droite : il doit parfois choisir selon une condition (un élève est-il admis ? sa moyenne est-elle ≥ 10 ?).

La structure **Si...Alors...Sinon...FinSi** teste une condition booléenne :

```

Si moyenne >= 10 Alors
    Ecrire("Admis")
Sinon
    Ecrire("Refuse")
FinSi

```

La partie **Sinon** est facultative : si l'on n'a rien à faire quand la condition est fausse, on l'omet.

La structure **Selon...FinSelon** évite les **Si** imbriqués quand on teste plusieurs valeurs d'une même variable :

```

Selon mention Faire
    1 : Ecrire("Passable")
    2 : Ecrire("Assez bien")
    3 : Ecrire("Bien")
    4 : Ecrire("Tres bien")
    Sinon Ecrire("Mention inconnue")
FinSelon

```

1.6 Rappel des structures itératives

Pour traiter 500 élèves, on n'écrit pas 500 fois la même instruction : on utilise une **boucle**, qui répète un bloc d'instructions. Il existe trois formes à connaître parfaitement.

1. La boucle Pour...FinPour : on connaît à l'avance le nombre de répétitions. Un compteur varie automatiquement d'une valeur initiale à une valeur finale.

```

Pour i ← 1 à 500 Faire
    Ecrire("Moyenne de l'eleve ", i, " : ")
    Lire(moy)
FinPour

```

2. La boucle TantQue...FinTantQue : on répète tant qu'une condition est vraie. Le test se fait avant chaque itération : le corps peut ne jamais s'exécuter.

```
TantQue moy <> -1 Faire      // -1 = valeur sentinelle d'arrêt
    somme ← somme + moy
    Lire(moy)
FinTantQue
```

3. La boucle Répéter...Jusqu'à : le corps s'exécute au moins une fois, puis on recommence jusqu'à ce que la condition devienne vraie (le test se fait après).

```
Repeter
    Ecrire("Saisir une moyenne entre 0 et 20 : ")
    Lire(moy)
Jusqu'à (moy >= 0) Et (moy <= 20)
```

MÉTHODE Bien choisir entre Pour et TantQue

1. Pose-toi la question : « le nombre de répétitions est-il connu avant d'entrer dans la boucle ? »
2. Si oui (parcourir 500 élèves, calculer la somme des 100 premiers entiers) : utilise **Pour**. Le compteur est géré automatiquement, impossible de l'oublier.
3. Si non (lire des moyennes jusqu'à la saisie de -1, chercher un élève et s'arrêter dès qu'on l'a trouvé) : utilise **TantQue** avec une condition d'arrêt claire.
4. Réserve **Répéter...Jusqu'à** aux cas où le corps doit s'exécuter au moins une fois, typiquement la saisie contrôlée d'une valeur.
5. Vérifie toujours que la condition du **TantQue** finira par devenir fausse, sinon la boucle est infinie.

1.7 Compteur et accumulateur : les deux piliers des boucles

Deux variables jouent des rôles stéréotypés dans presque toutes les boucles :

- le **compteur** compte le nombre de passages : initialisé à 0 (ou géré par le **Pour**), il est incrémenté de 1 à chaque itération ($\text{cpt} \leftarrow \text{cpt} + 1$);
- l'**accumulateur** cumule des valeurs : initialisé à 0 pour une somme (à 1 pour un produit), il est enrichi à chaque passage ($\text{somme} \leftarrow \text{somme} + \text{moy}$).

ATTENTION Variable non initialisée : l'erreur classique

Écrire $\text{somme} \leftarrow \text{somme} + \text{moy}$ sans avoir fait $\text{somme} \leftarrow 0$ avant la boucle est l'erreur la plus fréquente. La variable contient alors une valeur indéterminée (« résidu mémoire »), et le

résultat final est faux et imprévisible. Règle d'or : tout compteur et tout accumulateur doit être initialisé AVANT la boucle, jamais dedans (sinon il serait remis à zéro à chaque tour!).

1.8 Exemple chiffré : somme des N premiers entiers

EXERCICE RÉSOLU 1 Calcul de $1 + 2 + \dots + N$ avec trace d'exécution

Énoncé. Écrire un algorithme qui lit un entier N strictement positif et calcule la somme $S = 1 + 2 + \dots + N$. Donner la trace d'exécution pour $N = 4$.

SOLUTION

Solution. On utilise un accumulateur S initialisé à 0 et un compteur i géré par la boucle Pour.

```

Algorithme SommePremiersEntiers
Variables N, i, S : Entier
Debut
    Ecrire("Entrer N : ")
    Lire(N)
    S ← 0                // initialisation de l'accumulateur
    Pour i ← 1 à N Faire
        S ← S + i        // accumulation
    FinPour
    Ecrire("La somme vaut ", S)
Fin
  
```

Trace d'exécution pour $N = 4$ (table d'exécution simplifiée, boucle déroulée) :

Ligne	Instruction	i	S	Écran
1	Lire(N)	-	-	Entrer N : 4
2	$S \leftarrow 0$	-	0	
3	$S \leftarrow S + i$	1	1	
4	$S \leftarrow S + i$	2	3	
5	$S \leftarrow S + i$	3	6	
6	$S \leftarrow S + i$	4	10	
7	Écrire(S)	4	10	La somme vaut 10

Vérification : $1 + 2 + 3 + 4 = 10$, conforme à la formule $\frac{N(N+1)}{2} = \frac{4 \times 5}{2} = 10$. La trace confirme que l'accumulateur a bien été initialisé à 0 et incrémenté dans le bon ordre.

À RETENIR L'essentiel de la section

Un algorithme comporte trois parties : l'en-tête, la partie déclarative (constantes et variables typées) et le corps entre `Début` et `Fin`. Les instructions simples sont la lecture (`Lire`), l'écriture (`Écrire`) et l'affectation (`←`). Les structures alternatives (`Si...Alors...Sinon...FinSi`, `Selon...FinSelon`) permettent les choix; les structures itératives (`Pour`, `TantQue`, `Répéter...Jusqu'à`) permettent les répétitions. Tout compteur et tout accumulateur doit être initialisé avant la boucle. Ces fondations acquises, nous pouvons maintenant organiser les données en mémoire grâce aux structures de données, objet de la section suivante.

2 LES STRUCTURES DE DONNÉES

Dans la section précédente, nous avons revu la structure générale d'un algorithme (en-tête, déclarations, corps délimité par `Début` et `Fin`) ainsi que les instructions de base : affectation, lecture, écriture, structures alternatives et itératives. Jusqu'ici, nos algorithmes manipulaient quelques **variables simples** : un âge, une moyenne, un compteur. Cela suffit pour de petits problèmes. Mais imagine maintenant la situation suivante, bien réelle dans nos lycées : le proviseur du lycée de Ngaoundéré veut un programme qui calcule la moyenne de classe de chacun des 60 élèves de la Terminale C, puis affiche le tableau des mérites. Faut-il déclarer 60 variables `note1`, `note2`, ..., `note60`? Et pour les 800 élèves de l'établissement? Évidemment non. Il faut un moyen d'organiser les données : c'est exactement le rôle des **structures de données**, que nous étudions dans cette section.

2.1 Notion de structure de données

DÉFINITION Structure de données

Une **structure de données** est un ensemble organisé de données, regroupées sous un même nom et accessibles selon des règles précises (mode d'accès, ordre de lecture, opérations autorisées). Le choix d'une structure de données dépend du problème à résoudre : nombre de données, nature des données, opérations fréquentes à effectuer.

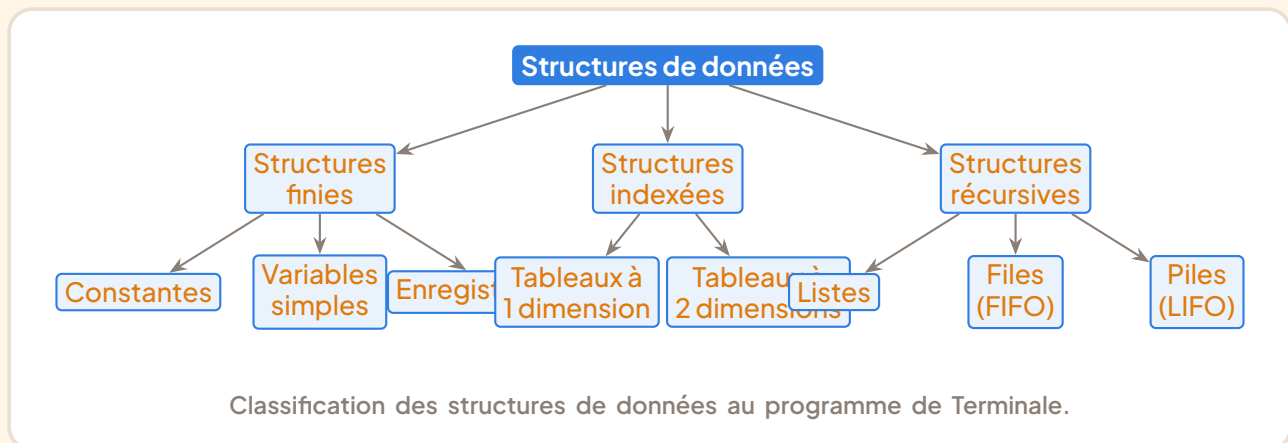
L'idée est toute simple : de la même façon qu'un bibliothécaire ne jette pas les livres en vrac dans une salle mais les range sur des étagères selon un classement, l'informaticien ne laisse pas ses données « en vrac » en mémoire : il les organise pour pouvoir les retrouver, les parcourir et les modifier efficacement. Une structure de données se caractérise donc par deux choses :

- le **contenu** : le type des données stockées (entiers, réels, caractères, ou données hétérogènes);
- les **règles d'accès** : comment atteindre un élément (directement par un indice? en parcou-

rant du début à la fin ? uniquement le premier arrivé ? uniquement le dernier empilé ?).

Le programme officiel distingue trois grandes familles de structures de données :

1. les **structures finies** : constantes, variables simples et enregistrements ;
2. les **structures indexées** : les tableaux ;
3. les **structures récursives** : les listes, les files et les piles.



2.2 Les structures finies

DÉFINITION Structure finie

Une **structure finie** est une structure de données dont le nombre d'éléments est fixe et connu à l'avance, et dont chaque élément est accessible par son nom. On y trouve les constantes, les variables simples et les enregistrements.

La constante. Une **constante** est une donnée dont la valeur est fixée à la déclaration et ne peut jamais être modifiée pendant l'exécution. Elle sert à donner un nom à une valeur de référence : le taux de TVA, le nombre de notes d'un bulletin, la valeur de π .

La variable simple. Une **variable simple** est une zone mémoire nommée, contenant une seule valeur d'un type donné (entier, réel, caractère, booléen, chaîne), valeur qui peut changer au cours de l'exécution grâce à l'affectation.

L'enregistrement. Jusqu'ici, tout cela te est familier. La vraie nouveauté est l'**enregistrement**. Observe la fiche d'un élève au secrétariat : elle contient un nom (chaîne de caractères), une date de naissance (chaîne), une classe (chaîne) et une moyenne (réel). Ces informations sont de **types différents**, mais elles décrivent un même objet : l'élève. Il serait absurde de les éparpiller dans quatre variables sans lien entre elles.

DÉFINITION Enregistrement

Un **enregistrement** est une structure finie qui regroupe sous un même nom plusieurs données de types éventuellement différents, appelées **champs**. Chaque champ est accessible par son nom, selon la syntaxe : `nomEnregistrement.nomChamp`.

PROPRIÉTÉ Déclaration et accès à un enregistrement

En écriture algorithmique, on déclare d'abord le type enregistrement, puis les variables de ce type :

```
Type
  Eleve = Enregistrement
    nom    : Chaîne
    classe : Chaîne
    moyenne : Reel
  FinEnregistrement
Variable
  e : Eleve
```

L'accès à un champ se fait avec la notation pointée : `e.nom` désigne le champ `nom` de l'enregistrement `e`. On peut lire, écrire ou affecter chaque champ comme une variable ordinaire : `e.moyenne ← 14,5`.

2.3 Les structures indexées : les tableaux

Revenons à nos 60 élèves de Terminale C. Leurs notes d'informatique sont **toutes de même type** (des réels) et elles sont **nombreuses**. L'enregistrement ne convient pas : il sert à regrouper des données différentes décrivant un objet. Ici, c'est l'inverse : des données identiques en nature, en grand nombre. Il faut une structure indexée : le **tableau**.

DÉFINITION Structure indexée — tableau

Une **structure indexée** est une structure de données regroupant un nombre fixe d'éléments de même type, repérés par leur position, appelée **indice**. Le **tableau** est la structure indexée de base : l'accès à un élément est direct grâce à son indice, selon la syntaxe `nomTableau[i]`.

On peut se représenter un tableau comme une rangée de casiers numérotés : tous les casiers ont la même taille (même type de contenu), et pour ouvrir un casier, il suffit de connaître son numéro — inutile d'ouvrir les précédents. C'est ce qu'on appelle l'**accès direct** (ou accès indexé).

PROPRIÉTÉ Déclaration d'un tableau à une dimension

En écriture algorithmique :

```
Variable  
notes : Tableau[1..60] de Reel
```

Cette déclaration réserve 60 cases mémoire contiguës, nommées `notes[1]`, `notes[2]`, ..., `notes[60]`. L'élément d'indice i s'utilise comme une variable ordinaire : `notes[i] ← 12,5` ou `Ecrire(notes[i])`.

ATTENTION Tableau ≠ liste : accès direct contre accès séquentiel

Ne confonds jamais un tableau et une liste. Dans un tableau, l'accès est direct : `T[37]` donne immédiatement le 37^{ème} élément, sans toucher aux 36 premiers. Dans une liste, l'accès est séquentiel : pour atteindre le 37^{ème} élément, il faut obligatoirement partir du premier et suivre les chaînons un à un. Conséquence pratique : le tableau est idéal quand on connaît la position des données ; la liste est souple quand le nombre de données varie constamment, mais la recherche y est plus lente.

Mention du tableau à deux dimensions. Pour représenter les notes de plusieurs élèves dans plusieurs matières, on utilise un tableau à deux dimensions, analogue à une feuille de tableur : `bulletin : Tableau[1..60, 1..8] de Reel`. L'élément `bulletin[i, j]` est alors la note de l'élève i dans la matière j . Nous y reviendrons dans les sections suivantes ; retiens simplement qu'il faut deux indices : une ligne et une colonne.

2.4 Les structures récursives : listes, files et piles

Certaines structures ne sont pas de taille fixe : elles grandissent et rétrécissent pendant l'exécution, chaque élément « pointant » vers le suivant. On les appelle **structures récursives**, car chaque élément contient (ou référence) un élément de même nature que lui : une liste, c'est un premier élément suivi... d'une liste ! Au programme de Terminale, on en attend une présentation intuitive : comprendre le principe et les règles d'accès, sans entrer dans la programmation des pointeurs.

DÉFINITION Structure récursive

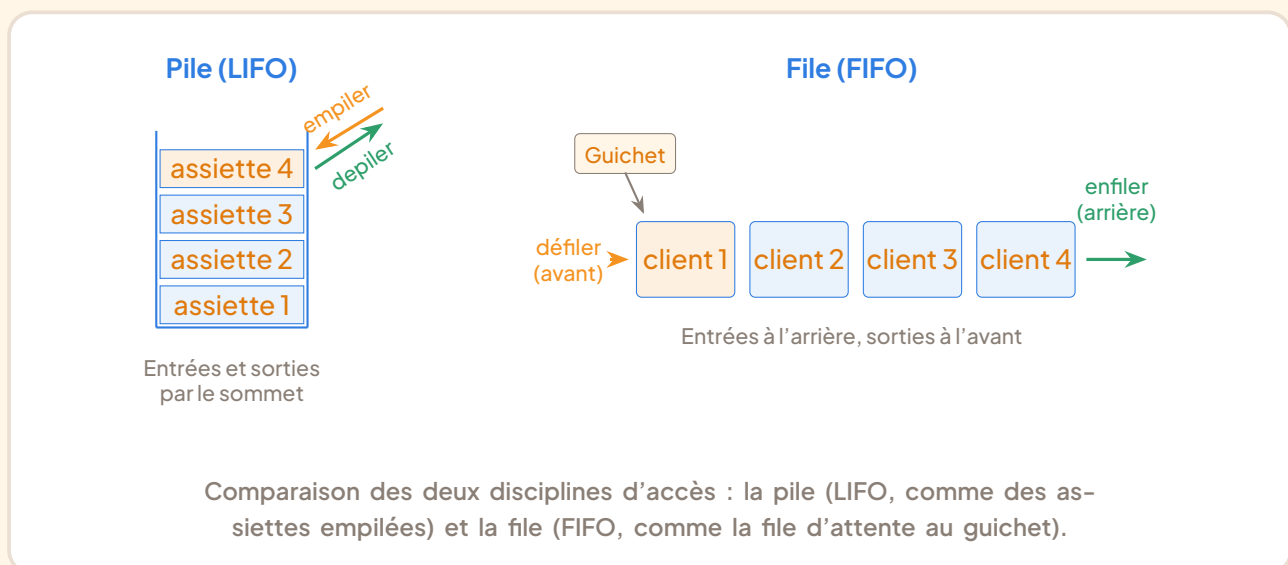
Une **structure récursive** est une structure de données de taille variable, formée d'éléments (appelés maillons ou nœuds) dont chacun contient une donnée et une référence vers l'élément suivant. L'accès y est séquentiel : on parcourt la chaîne maillon par maillon. Les trois structures récursives classiques sont la liste, la file et la pile.

La liste. Une **liste** est une suite chaînée d'éléments : chaque maillon connaît le suivant. On peut insérer ou supprimer un élément à n'importe quelle position sans déplacer les autres — contrai-

rement au tableau, où supprimer la case 3 oblige à « tasser » toutes les suivantes. En contrepartie, pour atteindre un élément, il faut parcourir la liste depuis le début.

La pile (LIFO). Une **pile** fonctionne comme une pile d'assiettes au restaurant : on empile une assiette au sommet, et on dépile toujours l'assiette du sommet. La règle est **LIFO** (Last In, First Out) : le **dernier entré est le premier sorti**. Deux opérations seulement sont permises : **empiler** (ajouter au sommet) et **dépiler** (retirer le sommet). C'est le mécanisme utilisé par le bouton « Annuler » (Ctrl+Z) de ton traitement de texte : la dernière action effectuée est la première annulée.

La file (FIFO). Une **file** fonctionne comme la file d'attente devant le guichet d'une agence Express Union : on arrive par l'arrière, on est servi à l'avant. La règle est **FIFO** (First In, First Out) : le **premier entré est le premier sorti**. Les deux opérations sont : **enfiler** (ajouter à l'arrière) et **défiler** (retirer à l'avant).



LESAVAIS-TU ? La file d'attente : une structure FIFO de la vie quotidienne

La file d'attente à la banque, à l'hôpital ou devant le guichet de la CAMTEL est exactement une structure FIFO : le premier client arrivé est le premier servi, et tout nouveau venu se place à l'arrière. Les informaticiens s'en inspirent partout : les documents envoyés à l'imprimante du cybercafé sont mis en file et imprimés dans l'ordre d'arrivée ; les appels vers le service client d'un opérateur mobile sont placés dans une file (« vous êtes le 3^{ème} appel en attente... »). À l'inverse, l'historique de navigation de ton navigateur (bouton « Précédent ») est une pile LIFO : la dernière page visitée est la première à laquelle on revient.

2.5 Comment choisir la bonne structure de données ?

Face à un problème concret, le choix de la structure est une étape décisive de la résolution : un mauvais choix rend l'algorithme compliqué, lent, voire faux. Trois questions guident ce choix.

MÉTHODE**Choisir une structure de données adaptée au problème**

1. Les données sont-elles homogènes (même type) ou hétérogènes (types différents) ?
 - Hétérogènes et décrivant un même objet (nom, âge, moyenne d'un élève) → enregistrement.
 - Homogènes et nombreuses (les 60 notes d'une classe) → tableau.
2. Le nombre de données est-il fixe et connu à l'avance ?
 - Oui → structure finie ou tableau (taille fixée à la déclaration).
 - Non, il varie pendant l'exécution → structure récursive (liste, file, pile).
3. Quel mode d'accès le problème impose-t-il ?
 - Accès direct à l'élément de rang i → tableau.
 - Traitement dans l'ordre d'arrivée (premier arrivé, premier servi) → file (FIFO).
 - Traitement en ordre inverse (dernier arrivé, premier servi : annulations, retour en arrière) → pile (LIFO).
 - Insertions et suppressions fréquentes en plein milieu → liste.

Structure	Type des éléments	Taille	Mode d'accès
Variable simple	Une seule valeur	Fixe (1)	Par le nom
Enregistrement	Hétérogènes (champs)	Fixe	Par le nom du champ
Tableau	Homogènes	Fixe	Direct, par indice
Liste	Homogènes	Variable	Séquentiel, maillon par maillon
File (FIFO)	Homogènes	Variable	Premier entré, premier sorti
Pile (LIFO)	Homogènes	Variable	Dernier entré, premier sorti

EXEMPLE**Modéliser les données d'un élève et d'une classe**

Le secrétariat du lycée veut informatiser ses bulletins. Deux besoins distincts apparaissent :

- la fiche d'un élève (nom, classe, moyenne) : données de types différents décrivant un seul objet → un enregistrement ;
- les 6 notes d'informatique d'un élève : données de même type, en nombre fixe → un tableau.

```

Type
  Eleve = Enregistrement
    nom   : Chaîne
    classe : Chaîne
    moyenne : Reel
  FinEnregistrement
Variable
  e : Eleve // la fiche d'un élève
  notes : Tableau[1..6] de Reel // ses 6 notes d'informatique
Debut
  e.nom ← "MBALLA Amina"
  e.classe ← "Tle C"
  notes[1] ← 15,5 // accès direct par l'indice
  e.moyenne ← notes[1] // un champ s'utilise comme une variable
  Ecrire(e.nom, " a obtenu ", notes[1], " au devoir 1")
Fin

```

À l'écran : MBALLA Amina a obtenu 15,5 au devoir 1. Remarque la différence d'accès : `e.nom` utilise le nom du champ (structure finie), `notes[1]` utilise l'indice (structure indexée).

EXERCICE RÉSOLU 2 Quelle structure pour quel problème ?

Énoncé. Pour chacune des situations suivantes, indique la structure de données la plus adaptée et justifie ton choix en une phrase :

1. mémoriser les températures relevées chaque jour du mois de mars à Yaoundé ;
2. mémoriser la fiche d'un livre de la bibliothèque scolaire (titre, auteur, année, cote) ;
3. gérer les élèves qui se présentent au comptoir de la cantine ;
4. gérer l'historique des pages visitées dans un navigateur (bouton « Précédent ») ;
5. mémoriser les notes des 8 matières pour les 60 élèves de la classe.

SOLUTION

Solution détaillée.

1. Tableau `temp` : Tableau[1..31] de Reel : 31 données homogènes, en nombre fixe et connu ; l'indice correspond naturellement au jour du mois (accès direct : `temp[15]` = température du 15 mars).
2. Enregistrement : les quatre informations sont de types différents (chaînes et entier) et décrivent un même objet, le livre ; chaque champ sera accessible par son nom (`livre.titre`).
3. File (FIFO) : à la cantine, le premier élève arrivé doit être le premier servi ; tout nouvel arrivant se place à l'arrière. C'est la discipline « premier entré, premier sorti ».
4. Pile (LIFO) : le bouton « Précédent » ramène d'abord à la dernière page visitée ; c'est la discipline « dernier entré, premier sorti ».

5. Tableau à deux dimensions `notes` : `Tableau[1..60, 1..8]` de `Reel` : données homogènes en grand nombre, repérées par deux indices (l'élève i et la matière j), comme une feuille de tableur.

À RETENIR L'essentiel de la section

- Une **structure de données** est un ensemble organisé de données accessibles selon des règles précises.
- Structures finies (nombre fixe, accès par nom) : constantes, variables simples, enregistrements (champs de types différents, accès par `enregistrement.champ`).
- Structures indexées : les tableaux, éléments de même type en nombre fixe, accès direct par indice `T[i]` ; le tableau à deux dimensions utilise deux indices `T[i, j]`.
- Structures récursives (taille variable, accès séquentiel) : la liste (chaînage de maillons), la file (FIFO : premier entré, premier sorti) et la pile (LIFO : dernier entré, premier sorti).
- Le choix d'une structure se fait en trois questions : homogène ou hétérogène ? taille fixe ou variable ? accès direct, ordre d'arrivée ou ordre inverse ?

Maintenant que le paysage des structures de données est clair, la section suivante se concentrera sur les deux structures au cœur du programme de Terminale : le tableau et l'enregistrement, que nous apprendrons à déclarer, remplir et parcourir dans des algorithmes complets.

3 TABLEAUX ET ENREGISTREMENTS

Dans la section précédente, nous avons classé les structures de données en trois grandes familles : les structures finies (constantes, variables, enregistrements), les structures indexées (tableaux) et les structures récursives (listes, piles, files). Nous allons maintenant étudier en profondeur les deux structures que le programme de Terminale met réellement en œuvre dans les algorithmes : le **tableau** et l'**enregistrement**. Ce sont elles qui permettent de manipuler, dans un algorithme, des ensembles de données de la vie réelle : les 10 notes d'un élève, la liste des 45 élèves d'une classe de Terminale C du lycée de Nkongsamba, le stock d'une boutique.

3.1 Le tableau : une variable à plusieurs cases

Le problème à résoudre. Imaginons que le professeur d'informatique veuille écrire un algorithme qui calcule la moyenne des 10 notes d'informatique d'un élève. Avec les connaissances des sections précédentes, il faudrait déclarer dix variables : `n1, n2, ..., n10`, écrire dix instructions `Lire`, puis une addition à dix termes. Et si l'on passait à 60 notes ? À 500 élèves ? Cette approche est impraticable. Il faut une structure capable de regrouper plusieurs valeurs de même type sous un seul nom, chaque valeur étant repérée par sa position.

DÉFINITION Tableau

Un **tableau** est une structure de données indexée qui regroupe, sous un même nom, un nombre fixe d'éléments de même type, chaque élément étant repéré par un numéro de position appelé **indice**. Le nombre d'éléments du tableau est sa **taille** (ou dimension), fixée à la déclaration et non modifiable ensuite.

PROPRIÉTÉ Déclaration d'un tableau en écriture algorithmique

La déclaration d'un tableau se fait dans la partie déclarative de l'algorithme, avec la syntaxe :

Tableau T[1..N] de type

où T est le nom du tableau, N sa taille (constante entière) et type le type commun de tous les éléments (entier, réel, chaîne, caractère, booléen). L'intervalle 1..N signifie que les indices valides vont de 1 à N inclus. Ainsi, pour les 10 notes de notre élève :

Constante N = 10
Tableau Notes[1..N] de reel

ATTENTION Convention sur le premier indice

En écriture algorithmique au lycée, on fait démarrer les indices à 1 : le premier élément est T[1] et le dernier T[N]. Attention : en langage C, que vous étudierez plus tard, le premier indice est 0 et le dernier N-1. Retenez la convention de chaque langage et ne les mélangez jamais.

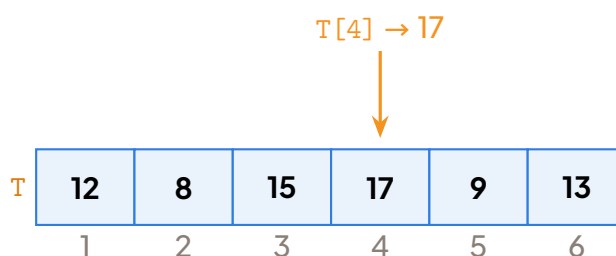
Accès à un élément. Toute la puissance du tableau vient de l'accès direct à chaque case grâce à son indice, entre crochets.

PROPRIÉTÉ Accès à l'élément d'indice i

Si T est un tableau et i un entier tel que $1 \leq i \leq N$, alors T[i] désigne l'élément d'indice i et s'utilise exactement comme une variable ordinaire :

- en écriture (affectation) : $T[i] \leftarrow \text{valeur}$;
- en lecture (dans une expression ou un affichage) : $\text{Ecrire}(T[i])$ ou $S \leftarrow S + T[i]$.

L'indice peut être une constante (T[4]), une variable (T[i]) ou toute expression entière (T[i+1]), ce qui permet de parcourir le tableau avec une boucle.



Représentation mémoire du tableau $T[1..6]$: chaque case contient une valeur et est repérée par son indice. L'expression $T[4]$ désigne directement la case d'indice 4, qui contient 17.

ATTENTION Dépassement d'indice : l'erreur classique

Accéder à $T[0]$ ou à $T[N+1]$ est une erreur : ces cases n'existent pas. En algorithmique, c'est une faute de raisonnement ; en langage C, le programme lit ou écrit dans une zone mémoire qui ne lui appartient pas, avec des résultats imprévisibles (valeurs absurdes, plantage). Avant tout accès $T[i]$, vérifie mentalement que $1 \leq i \leq N$. Piège typique : écrire Pour i de 0 à N au lieu de Pour i de 1 à N .

3.2 Remplir et parcourir un tableau

Remplissage par une boucle Pour. Pour saisir les N éléments d'un tableau, on ne répète pas N fois l'instruction Lire : on utilise une boucle Pour dont le compteur i sert d'indice.

```
Pour i de 1 à N Faire
    Ecrire("Entrez l'élément ", i, " : ")
    Lire(T[i])
FinPour
```

Au premier tour, i vaut 1 et l'utilisateur remplit $T[1]$; au deuxième tour, $T[2]$, et ainsi de suite jusqu'à $T[N]$. Une seule instruction Lire suffit donc, quelle que soit la taille du tableau.

MÉTHODE Schéma type du parcours complet d'un tableau

Tout traitement qui doit s'appliquer à tous les éléments d'un tableau $T[1..N]$ suit le même gabarit, à apprendre par cœur :

1. Initialiser les variables de travail avant la boucle (somme à 0, maximum à $T[1]$, compteur à 0) ;
2. Parcourir : Pour i de 1 à N Faire ;
3. Traiter l'élément courant $T[i]$ dans le corps de la boucle (l'afficher, l'additionner, le comparer, le compter) ;
4. Exploiter le résultat après la boucle (afficher la somme, calculer la moyenne, etc.).

Affichage de tous les éléments. Application immédiate du schéma :

```
Ecrire("Contenu du tableau :")
Pour i de 1 a N Faire
    Ecrire("T[" + i + "] = ", T[i])
FinPour
```

Parcours avec traitement. Voici les quatre traitements classiques exigibles au Baccalauréat, tous fondés sur le même gabarit.

```
// Somme des elements
S <- 0
Pour i de 1 a N Faire
    S <- S + T[i]
FinPour

// Moyenne (N > 0)
Moy <- S / N

// Maximum
Max <- T[1]
Pour i de 2 a N Faire
    Si T[i] > Max Alors
        Max <- T[i]
    FinSi
FinPour

// Comptage : nombre d'elements superieurs a 10
Cpt <- 0
Pour i de 1 a N Faire
    Si T[i] > 10 Alors
        Cpt <- Cpt + 1
    FinSi
FinPour
```

Notez bien les initialisations : la somme et le compteur partent de 0 ; le maximum part de $T[1]$ (le premier élément réel), et non de 0 — sinon un tableau ne contenant que des valeurs négatives donnerait un maximum faux.

EXERCICE RÉSOLU 3 Moyenne de 10 notes

Énoncé. Écrire un algorithme qui demande à l'utilisateur les 10 notes d'un élève (sur 20), les stocke dans un tableau, puis calcule et affiche leur somme et leur moyenne.

SOLUTION

Solution détaillée. On applique le schéma type : déclaration du tableau, boucle de remplissage, puis boucle de cumul.

```
Algorithme MoyenneNotes
Constante N = 10
Variable
    Tableau Notes[1..N] de reel
    i : entier
```

```

S, Moy : reel
Debut
// 1. Remplissage du tableau
Pour i de 1 a N Faire
    Ecrire("Note numero ", i, " : ")
    Lire(Notes[i])
FinPour
// 2. Calcul de la somme
S <- 0
Pour i de 1 a N Faire
    S <- S + Notes[i]
FinPour
// 3. Calcul et affichage de la moyenne
Moy <- S / N
Ecrire("Somme = ", S)
Ecrire("Moyenne = ", Moy)
Fin

```

Vérification mentale : si toutes les notes valent 12, alors $S = 10 \times 12 = 120$ et $Moy = 120/10 = 12$: le résultat est cohérent.

3.3 L'enregistrement : regrouper des informations de types différents

Le problème à résoudre. Le tableau impose que tous ses éléments soient de même type. Or, pour décrire un élève de Terminale, on a besoin simultanément de son nom (chaîne), de son âge (entier) et de sa moyenne (réel). Trois variables séparées `nom`, `age`, `moyenne` ne disent pas qu'elles décrivent le même élève. Il faut une structure qui regroupe des données **de types différents** en un seul objet.

DÉFINITION Enregistrement et champ

Un **enregistrement** est une structure de données finie qui regroupe sous un même nom plusieurs données appelées **champs**, pouvant être de types différents. Chaque champ possède un nom et un type. L'enregistrement est défini comme un nouveau type, puis on déclare des variables de ce type.

PROPRIÉTÉ Déclaration d'un type enregistrement et accès aux champs

La définition du type se fait dans la partie déclarative, avant les variables :

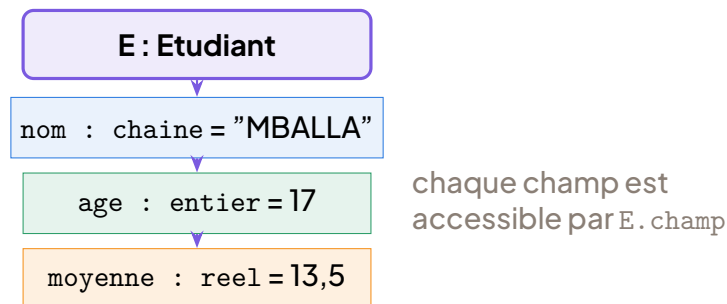
```

Type Etudiant = Enregistrement
    nom      : chaîne
    age      : entier
    moyenne  : reel
FinEnregistrement

```

On déclare ensuite une variable de ce type : `E : Etudiant`. L'accès à chaque champ se

fait par la notation pointée : `E.nom`, `E.age`, `E.moyenne`. Chaque champ s'utilise comme une variable de son type, en lecture comme en écriture : `E.age ← 17`, `Lire(E.nom)`, `Ecrire(E.moyenne)`.



Un enregistrement `E` de type `Etudiant` : une seule variable, trois champs de types différents.

EXEMPLE Un élève de Terminale en algorithme

Algorithme complet qui saisit puis affiche la fiche d'un élève :

```

Algorithme FicheEleve
Type Etudiant = Enregistrement
  nom   : chaine
  age    : entier
  moyenne : reel
FinEnregistrement
Variable
  E : Etudiant
Debut
  Ecrire("Nom de l'eleve : ")
  Lire(E.nom)
  Ecrire("Age : ")
  Lire(E.age)
  Ecrire("Moyenne generale : ")
  Lire(E.moyenne)
  // Affichage de la fiche
  Ecrire("--- Fiche de l'eleve ---")
  Ecrire("Nom   : ", E.nom)
  Ecrire("Age    : ", E.age, " ans")
  Ecrire("Moyenne : ", E.moyenne, " / 20")
  Si E.moyenne >= 10 Alors
    Ecrire(E.nom, " est admis(e).")
  Sinon
    Ecrire(E.nom, " doit redoubler d'efforts.")
  FinSi
Fin
  
```

Remarque : `E.moyenne >= 10` montre qu'un champ s'utilise dans n'importe quelle expression, comme une variable ordinaire.

ATTENTION Ne pas confondre le type et la variable

`Etudiant` est un type (un moule) : on ne peut pas écrire `Etudiant.nom`. C'est la variable `E`, déclarée de type `Etudiant`, qui possède des champs : on écrit `E.nom`. Autre piège : oublier le point et écrire `Enom` — l'algorithme chercherait alors une variable nommée `Enom`, qui n'existe pas.

LESAVAIS-TU ? Hors-programme strict, mais utile pour le Bac : le tableau d'enregistrements

Pour gérer toute une classe, on combine les deux structures : un tableau d'enregistrements. On déclare `Tableau Classe[1..45]` de `Etudiant` : chaque case du tableau est un enregistrement complet. L'élève d'indice `i` s'appelle alors `Classe[i].nom` et sa moyenne `Classe[i].moyenne`. C'est exactement le principe des logiciels de gestion des notes utilisés dans les lycées camerounais : un tableau de lignes, chaque ligne étant une fiche élève. Cette combinaison n'est pas exigible en écriture algorithmique, mais la comprendre éclaire les exercices de synthèse.

EXERCICE 1 Stock d'une boutique

Une boutique de Douala vend 8 articles. Écrire un algorithme qui : (a) saisit dans un tableau les quantités en stock des 8 articles ; (b) affiche les quantités supérieures ou égales à 20 ; (c) affiche la quantité totale en stock et le nombre d'articles en rupture (quantité nulle).

CORRIGÉ Exercice : stock d'une boutique

```

Algorithme StockBoutique
Constante N = 8
Variable
    Tableau Q[1..N] de entier
    i, Total, Ruptures : entier
Debut
    Pour i de 1 a N Faire
        Ecrire("Quantite de l'article ", i, " : ")
        Lire(Q[i])
    FinPour
    Ecrire("Articles bien approvisionnes :")
    Pour i de 1 a N Faire
        Si Q[i] >= 20 Alors
            Ecrire("Article ", i, " : ", Q[i])
        FinSi
    FinPour
    Total <- 0

```

```
Ruptures <- 0
Pour i de 1 a N Faire
    Total <- Total + Q[i]
    Si Q[i] = 0 Alors
        Ruptures <- Ruptures + 1
    FinSi
FinPour
Ecrire("Stock total : ", Total)
Ecrire("Articles en rupture : ", Ruptures)
Fin
```

On remarque que les deux derniers traitements (somme et comptage) sont fusionnés dans une seule boucle : c'est plus efficace, et cela reste le schéma type du parcours complet.

À RETENIR L'essentiel de la section

- Un **tableau** $T[1..N]$ regroupe N valeurs de même type ; $T[i]$ accède directement à l'élément d'indice i , en lecture comme en écriture, avec $1 \leq i \leq N$.
- Tout parcours suit le même schéma : initialisation, boucle Pour i de 1 à N , traitement de $T[i]$, exploitation du résultat.
- Un **enregistrement** regroupe des champs de types différents ; on définit le type, on déclare une variable, et on accède aux champs par la notation pointée $E.champ$.
- Erreur fatale à éviter : le dépassement d'indice ($T[0]$, $T[N+1]$).

Maintenant que nous savons stocker et parcourir des données, la section suivante répondra à une question naturelle : comment retrouver une valeur précise dans un tableau ? Ce sera la recherche séquentielle.

4 RECHERCHE SÉQUENTIELLE DANS UN TABLEAU

Nous savons désormais déclarer un tableau, le remplir et l'afficher élément par élément. Mais une question fondamentale se pose dès que l'on manipule des données : **comment retrouver une information précise au milieu d'une collection ?** Imagine un fichier d'élèves dans un établissement comme le Lycée de Nkolbisson : on a stocké les noms dans un tableau. Comment savoir si Kamga est inscrit, et à quelle ligne il se trouve, sans lire toute la liste manuellement ? C'est exactement le rôle de la **recherche séquentielle**, l'algorithme de base pour interroger un tableau non trié.

4.1 Problématique et principe de la recherche séquentielle

DÉFINITION Recherche séquentielle

Soit un tableau T de taille N (indices de 1 à N) et une valeur x à rechercher. La **recherche séquentielle** consiste à parcourir le tableau du premier au dernier élément, en comparant chaque valeur $T[i]$ à x , jusqu'à trouver une correspondance ou épuiser le tableau.

PROPRIÉTÉ Principe du témoin (drapeau)

Pour mémoriser le résultat de la recherche (trouvé ou non), on utilise une variable booléenne, appelée **témoin** ou **drapeau** (souvent nommée `trouve` ou `flag`).

- Initialisation : `trouve` $\leftarrow$ FAUX avant la boucle.
- Mise à jour : dès qu'on découvre $T[i] = x$, on affecte `trouve` $\leftarrow$ VRAI.
- Condition d'arrêt anticipé : on n'a plus besoin de continuer la boucle si `trouve` = VRAI.

4.2 Structure de la boucle : arrêt anticipé

Contrairement à un simple parcours complet (affichage, somme), la recherche peut s'arrêter avant la fin du tableau. La structure `TantQue` est idéale car elle permet de tester une condition composée à chaque itération.

PROPRIÉTÉ Condition de continuation de la boucle

La boucle se poursuit tant que :

1. On n'a pas dépassé la borne supérieure : $i \leq N$
2. ET on n'a pas encore trouvé la valeur : `non trouve` (ou `trouve` = FAUX)

Soit : `TantQue` ($i \leq N$) ET (NON `trouve`) Faire ... FinTantQue

ATTENTION Pièges classiques : initialisation et arrêt

- Oublier d'initialiser `trouve` à FAUX : si la variable contient une valeur aléatoire (résidu mémoire), l'algorithme peut croire avoir trouvé la valeur avant même de commencer.
- Utiliser `Si` au lieu de `TantQue` pour l'arrêt : écrire `Si` $T[i] = x$ Alors `trouve` $\leftarrow$ VRAI FinSi à l'intérieur d'une boucle `Pour` $i \leftarrow 1$ à N fonctionne, mais parcourt tout le tableau même après avoir trouvé. C'est correct mais inefficace. La boucle `TantQue` avec condition composée est la forme canonique enseignée au programme.
- Indice de début : au Cameroun, dans l'écriture algorithmique officielle (MINESEC), les

tableaux sont généralement indexés à partir de 1. Vérifie toujours l'énoncé : $T[1..N]$ ou $T[0..N-1]$.

4.3 Algorithme complet : Recherche Séquentielle

Voici l'écriture formelle, rigoureuse et commentée, conforme aux attendus du Baccalauréat.

```

Algorithme RechercheSequentielle
// But : Rechercher une valeur x dans un tableau T[1..N]
//      et afficher sa position si trouvée, sinon signaler l'absence.

Variable
  T : Tableau[1..N] d'entiers // N est une constante connue
  x, i : Entier
  trouve : Booleen
  pos : Entier                // Pour mémoriser la position trouvée

Début
  // 1. Saisie des données (tableau et valeur cherchée)
  //   (On suppose T déjà rempli ici, ou on ajoute une boucle de remplissage)
  Afficher("Entrez la valeur à rechercher : ")
  Saisir(x)

  // 2. Initialisation du témoin et de l'indice
  trouve <- FAUX
  i <- 1
  pos <- 0                    // 0 = position invalide (non trouvée)

  // 3. Parcours séquentiel avec arrêt anticipé
  TantQue (i <= N) ET (NON trouve) Faire
    Si T[i] = x Alors
      trouve <- VRAI
      pos <- i                // On mémorise l'indice où x a été trouvé
    Sinon
      i <- i + 1              // On passe à l'élément suivant
    FinSi
  FinTantQue

  // 4. Conclusion / Affichage du résultat
  Si trouve Alors
    Afficher("Valeur ", x, " trouvée à la position ", pos)
  Sinon
    Afficher("Valeur ", x, " absente du tableau")
  FinSi
Fin

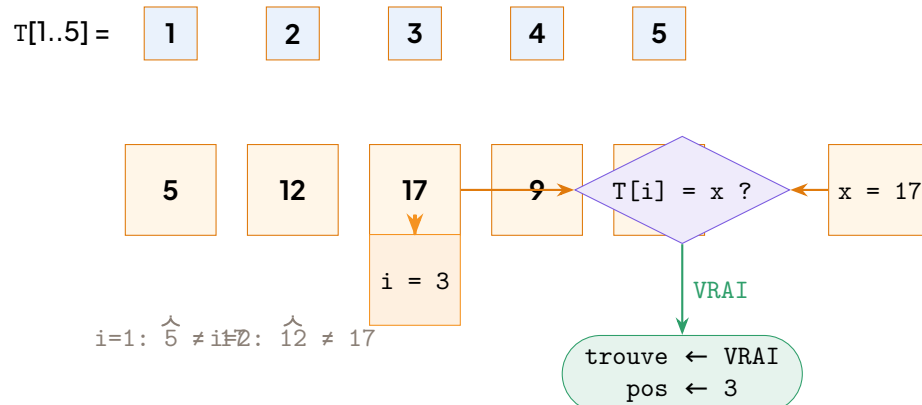
```

Listing 1 – Algorithme : Recherche séquentielle avec témoin

À RETENIR Points clés de l'algorithme

- La variable `pos` est optionnelle si on veut seulement savoir « trouvé/non trouvé », mais indispensable si l'énoncé demande « à quelle position ».

- L'incréméntation $i \leftarrow i + 1$ se fait uniquement dans le **Sinon**. Si on l'avait mise après le **FinSi** (hors alternative), on risquerait d'incrémenter i après avoir trouvé, perdant ainsi la position exacte (ou accédant hors limites si $i=N$).
- La condition $(i \leq N)$ ET (NON trouve) garantit qu'on ne sort jamais des bornes du tableau ($T[N+1]$ n'est jamais accédé).



Parcours séquentiel : le curseur i visite chaque case.
À $i=3$, la comparaison $T[3] == 17$ est vraie. Le témoin **trouve** passe à **VRAI**, la boucle s'arrête immédiatement.

Illustration du parcours séquentiel : recherche de 17 dans
 $T = [5, 12, 17, 9, 21]$. La boucle s'arrête à l'indice 3.

4.4 Cas particuliers et analyse du coût

EXEMPLE Cas particuliers : trace d'exécution (table d'exécution)

Recherchons $x = 17$ dans $T = [5, 12, 17, 9, 21]$ ($N=5$).

Itération	i	$T[i]$	Comparaison	trouve	Action / Commentaire
Init	-	-	-	FAUX	$i=1$, $pos=0$
1	1	5	$5 = 17?$ FAUX	FAUX	$i \leftarrow 2$
2	2	12	$12 = 17?$ FAUX	FAUX	$i \leftarrow 3$
3	3	17	$17 = 17?$ VRAI	VRAI	$pos \leftarrow 3$, ARRÊT boucle

Résultat final : **trouve** = **VRAI**, **pos** = 3. La valeur est en 3^{ème} position.

- **Valeur en première position ($i=1$)** : Une seule comparaison. Meilleur cas : $\Omega(1)$.

- **Valeur absente** : La boucle va jusqu'à $i = N$, fait N comparaisons, trouve reste FAUX. Pire cas : $O(N)$.
- **Occurrences multiples** : Cet algorithme (tel qu'écrit) s'arrête à la **première** occurrence trouvée (la plus petit indice). Si l'on veut toutes les positions, il faut retirer l'arrêt anticipé (boucle Pour complète) et stocker les indices dans un second tableau.

PROPRIÉTÉ Coût en comparaisons (admis)

Pour un tableau de taille N , l'algorithme effectue au maximum N comparaisons (cas où x est absent ou en dernière position).

- Meilleur cas : 1 comparaison ($x = T[1]$).
- Pire cas : N comparaisons (x absent ou $x = T[N]$).
- Moyenne (si x présent et équiprobable) : $\frac{N+1}{2}$ comparaisons.

On dit que la complexité en temps est linéaire : $O(N)$.

LESAVAIS-TU ? Le cahier d'appel du prof

Quand ton professeur appelle les noms un par un dans la liste de la classe pour savoir si tu es présent, il effectue une **recherche séquentielle** sur un « tableau » non trié (la liste d'appel). S'il trouve ton nom, il coche (témoin = VRAI) et passe au suivant. S'il arrive au bout sans trouver ton nom, tu es absent (témoin = FAUX). C'est exactement l'algorithme que nous venons d'écrire, exécuté par un humain !

4.5 Méthode : les 4 étapes pour écrire une recherche séquentielle

MÉTHODE Écrire un algorithme de recherche séquentielle

1. **Déclarer** : Tableau $T[1..N]$, valeur cherchée x , indice i , témoin trouve (Booleen), position pos (Entier).
2. **Initialiser** : trouve $\leftarrow$ FAUX, $i \leftarrow 1$, pos $\leftarrow 0$. Lire x (et remplir T si besoin).
3. **Parcourir et Comparer** : Boucle TantQue ($i \leq N$) ET (NON trouve) Faire
 Si $T[i] = x$ Alors trouve $\leftarrow$ VRAI; pos $\leftarrow i$
 Sinon $i \leftarrow i + 1$
 FinSi
 FinTantQue
4. **Conclure** : Si trouve Alors Afficher("Trouvé à", pos) Sinon Afficher("Absent")
 FinSi.

4.6 Exercice résolu : Recherche d'une note dans un bulletin

EXERCICE RÉSOLU 4 Recherche d'une note seuil

Énoncé : On dispose des notes de 30 élèves d'une classe de Terminale D stockées dans un tableau `NOTES[1..30]` (entiers sur 20). Écrire un algorithme qui permet de savoir s'il existe au moins une note égale à 20/20. Si oui, afficher « Excellence détectée » et le numéro de l'élève (son indice dans le tableau). Sinon afficher « Aucun 20/20 ».

SOLUTION

Solution détaillée

1. **Analyse :** C'est une recherche séquentielle classique où $x = 20$ est fixée (pas de saisie utilisateur). Le tableau est déjà rempli. On cherche la première occurrence de 20.
2. **Variables :** `NOTES[1..30]` : Entier, `i`, `pos` : Entier, `trouve` : Booleen.
3. **Algorithme :**

Algorithme DetectionExcellence

Constante

`TAILLE = 30`

Variable

`NOTES` : Tableau[1..TAILLE] d'entiers

`i`, `pos` : Entier

`trouve` : Booleen

Debut

// Remplissage simulé (dans la réalité, notes déjà en mémoire ou saisies)

Pour `i` <- 1 A TAILLE Faire

 Afficher("Note élève ", `i`, " : "); Saisir(`NOTES[i]`)

FinPour

// Recherche séquentielle de la valeur 20

`trouve` <- FAUX

`i` <- 1

`pos` <- 0

TantQue (`i` <= TAILLE) ET (NON `trouve`) Faire

 Si `NOTES[i] = 20` Alors

`trouve` <- VRAI

`pos` <- `i`

 Sinon

`i` <- `i` + 1

 FinSi

FinTantQue

// Résultat

Si `trouve` Alors

 Afficher("Excellence détectée ! Élève n°", `pos`, " a 20/20.")

Sinon

 Afficher("Aucun 20/20 dans cette classe.")

FinSi

Fin

Listing 2 – Algorithme : Détection Excellence

4. Trace partielle (exemple) : Si $NOTES = [14, 20, 18, 20, \dots]$.
 - $i=1 : 14 \neq 20 \rightarrow i=2$.
 - $i=2 : 20 = 20 \rightarrow trouve=VRAI, pos=2, ARRÊT$.
5. Sortie : Excellence détectée ! Élève n°2 a 20/20. (On ne voit pas le 20 de l'élève 4 car arrêt anticipé).

EXERCICE 2 À vous de jouer : Recherche dans un stock

Écrire un algorithme qui gère le stock d'une petite boutique à Yaoundé.

1. Déclarer un tableau $STOCK[1..50]$ d'entiers (quantités d'articles).
2. Remplir le tableau par saisie.
3. Demander à l'utilisateur un seuil critique S .
4. Rechercher s'il existe un article dont la quantité est strictement inférieure à S (risque de rupture).
5. Si trouvé : afficher « Alerte : Article n°[indice] en rupture (Qté = [valeur]) » pour le premier article concerné.
6. Sinon : afficher « Stock suffisant pour tous les articles ».

Indication : la condition de comparaison change ($<$ au lieu de $=$), mais la structure de l'algorithme reste identique.

CORRIGÉ Exercice : Recherche dans un stock

```

Algorithme AlerteRuptureStock
Constante MAX = 50
Variable
  STOCK : Tableau[1..MAX] d'entiers
  S, i, pos : Entier
  trouve : Booleen

Debut
  // 1. Remplissage
  Pour i <- 1 A MAX Faire
    Afficher("Quantité article ", i, " : "); Saisir(STOCK[i])
  FinPour

```

```
// 2. Saisie seuil
Afficher("Seuil critique d'alerte : "); Saisir(S)

// 3. Recherche première quantité < S
trouve <- FAUX
i <- 1
pos <- 0

TantQue (i <= MAX) ET (NON trouve) Faire
    Si STOCK[i] < S Alors // Condition changée ici
        trouve <- VRAI
        pos <- i
    Sinon
        i <- i + 1
    FinSi
FinTantQue

// 4. Affichage résultat
Si trouve Alors
    Afficher("ALERTE : Article n°", pos, " en rupture (Qté = ", STOCK[pos], ")")
Sinon
    Afficher("Stock suffisant pour tous les articles.")
FinSi
Fin
```

Note : On affiche `STOCK[pos]` pour donner la quantité exacte qui a déclenché l'alerte.

5 TRI PAR INSERTION

Dans la section précédente, nous avons appris à rechercher une valeur dans un tableau grâce à la recherche séquentielle. Nous avons remarqué au passage que cette recherche devient beaucoup plus efficace lorsque le tableau est **trié** : on peut alors s'arrêter dès qu'on dépasse la valeur cherchée, et des méthodes encore plus rapides deviennent possibles (recherche dichotomique). Mais comment obtenir un tableau trié ? C'est précisément l'objet de cette section : découvrir un premier **algorithme de tri**, le **tri par insertion**, qui permet d'ordonner les éléments d'un tableau, par exemple les notes d'une classe de la plus petite à la plus grande.

5.1 Le problème du tri

Situation. La fin du trimestre approche au lycée. Le professeur principal de la Terminale C a saisi dans un tableau les moyennes générales des 40 élèves, dans l'ordre de la liste de classe. Pour établir le tableau d'honneur, il veut afficher ces moyennes de la plus grande à la plus petite ; pour repérer les élèves en difficulté, il veut les voir de la plus petite à la plus grande. Dans les deux cas, il a besoin de **trier** le tableau.

DÉFINITION Tri d'un tableau

Trier un tableau, c'est réorganiser ses éléments de manière qu'ils apparaissent dans un ordre donné : **ordre croissant** (chaque élément est inférieur ou égal au suivant) ou **ordre décroissant** (chaque élément est supérieur ou égal au suivant). Un tableau T de N entiers est trié par ordre croissant lorsque :

$$T[1] \leq T[2] \leq T[3] \leq \dots \leq T[N]$$

EXEMPLE Tableau trié ou non ?

Le tableau $T = [1, 3, 5, 8]$ est trié par ordre croissant car $1 \leq 3 \leq 5 \leq 8$. Le tableau $T = [8, 3, 5, 1]$ n'est pas trié car $8 > 3$: dès le deuxième élément, l'ordre croissant est violé.

Il existe de nombreux algorithmes de tri (tri par sélection, tri à bulles, tri fusion, tri rapide...). Au programme de Terminale, nous étudions le **tri par insertion**, qui a l'immense avantage d'être celui que tout être humain utilise naturellement.

5.2 L'intuition : trier ses cartes en main

Imagine que tu joues aux cartes avec des amis dans la cour du lycée. On te distribue les cartes une à une. Comment fais-tu pour garder ta main triée ? Presque tout le monde procède ainsi :

1. la première carte est posée seule : une main d'une seule carte est évidemment triée ;
2. à chaque nouvelle carte reçue, tu la compares aux cartes déjà en main, de droite à gauche ;
3. tu déplaces vers la droite les cartes plus grandes qu'elle, puis tu l'**insères** à sa place, dans la zone déjà triée.

À aucun moment tu ne recommences tout depuis le début : tu exploites le fait que la partie gauche de ta main est déjà triée, et tu y insères la nouvelle carte. Le tri par insertion applique exactement cette stratégie à un tableau.

À RETENIR Idée fondamentale du tri par insertion

On parcourt le tableau de gauche à droite. À l'étape i (avec i allant de 2 à N) :

- la partie $T[1..i-1]$ est déjà triée (c'est la « main » du joueur) ;
- on prend la valeur $T[i]$ (la « nouvelle carte ») et on l'insère à sa place dans la partie triée, en décalant d'un cran vers la droite tous les éléments qui lui sont supérieurs.

5.3 Principe détaillé et variables clés

Formalisons l'insertion de l'élément d'indice i . Trois variables jouent un rôle essentiel :

Variable	Rôle
i	Frontière entre la partie triée (indices 1 à $i-1$) et la partie non encore traitée (indices i à N). C'est l'indice de l'élément à insérer.
x	Sauvegarde de la valeur $T[i]$ à insérer. Indispensable, car les décalages vont écraser la case $T[i]$.
j	Curseur de décalage : il parcourt la partie triée de droite à gauche (de $i-1$ vers 1) pour trouver la place de x .

L'insertion de x se déroule ainsi : tant que $j \geq 1$ et que $T[j] > x$, on recopie $T[j]$ dans $T[j+1]$ (décalage d'un cran vers la droite) et on recule j . Quand la boucle s'arrête, la case $T[j+1]$ est libre : on y place x .

ATTENTION Deux conditions obligatoires dans le TantQue

La condition de la boucle interne est `TantQue (j >= 1) ET (T[j] > x)`. Le test `j >= 1` est indispensable : sans lui, si x est plus petit que tous les éléments déjà triés, j deviendrait 0 et l'accès à $T[0]$ serait un débordement de tableau (indice hors limites). On teste toujours d'abord la validité de l'indice, ensuite la comparaison.

5.4 Écriture complète de l'algorithme

MÉTHODE Algorithme TriInsertion, commenté ligne par ligne

```

Algorithme TriInsertion
Variable
  T : Tableau[1..N] de Entier // tableau a trier
  i, j, x : Entier
Debut
  Pour i De 2 A N Faire           // T[1] seul est deja trie
    x <- T[i]                     // 1. sauvegarder la valeur a inserer
    j <- i - 1                    // 2. partir de la fin de la zone trie
    TantQue (j >= 1) ET (T[j] > x) Faire
      T[j+1] <- T[j]              // 3. decaler T[j] d'un cran a droite
      j <- j - 1                  // 4. reculer le curseur
    FinTantQue
    T[j+1] <- x                   // 5. inserer x a sa place
  FinPour
Fin

```

Commentaire des étapes clés :

1. Ligne 6 : la boucle externe commence à $i = 2$, car un tableau réduit à $T[1]$ est trivialement trié.
2. Ligne 7 : on sauvegarde $T[i]$ dans x avant tout décalage.
3. Lignes 9 à 12 : la boucle interne décale vers la droite tous les éléments de la partie triée

qui sont strictement supérieurs à x .

4. Ligne 13 : à la sortie de la boucle, tous les éléments d'indice 1 à j sont $\leq x$ et ceux d'indice $j + 2$ à i sont $> x$: la bonne case pour x est donc $j + 1$.

ATTENTION Erreur fréquente : écraser $T[i]$ avant de l'avoir sauvegardé

Beaucoup d'élèves écrivent directement les décalages sans passer par la variable x . Or le premier décalage $T[j+1] \leftarrow T[j]$ avec $j = i - 1$ recopie $T[i - 1]$ dans $T[i]$: la valeur initiale de $T[i]$ est alors définitivement perdue, et le tableau final contiendra un doublon et une valeur manquante. Règle d'or : toujours sauvegarder $x \leftarrow T[i]$ en premier.

5.5 Exécution pas à pas et table d'exécution

Le programme officiel exige de savoir exécuter un algorithme pas à pas à l'aide d'une **table d'exécution** (ou table de traçage). Cette table note, pour chaque instruction exécutée, les valeurs de toutes les variables et le contenu du tableau. C'est l'outil indispensable pour valider un algorithme et détecter les erreurs logiques.

MÉTHODE Construction d'une table d'exécution pour le tri par insertion

1. Colonnes : une colonne par variable (i, j, x) et une colonne par case du tableau ($T[1], T[2], \dots, T[N]$). On ajoute une colonne « Instruction » (numéro de ligne ou description).
2. Lignes : une nouvelle ligne à chaque modification d'une variable ou du tableau (affectation, incrémentation, entrée dans une boucle).
3. Initialisation : noter l'état initial du tableau avant le **Debut** de la boucle principale.
4. Boucle externe : pour chaque valeur de i , noter l'affectation de x et j .
5. Boucle interne : à chaque itération, noter le décalage ($T[j + 1] \leftarrow T[j]$) et la décrémentation de j .
6. Insertion : noter l'affectation finale $T[j + 1] \leftarrow x$.

EXEMPLE Table d'exécution complète pour $T = [8, 3, 5, 1]$ ($N = 4$)

Instr.	<i>i</i>	<i>j</i>	<i>x</i>	<i>T</i> [1]	<i>T</i> [2]	<i>T</i> [3]	<i>T</i> [4]
Init	-	-	-	8	3	5	1
<i>i</i> ← 2	2	-	-	8	3	5	1
<i>x</i> ← <i>T</i> [2]	2	-	3	8	3	5	1
<i>j</i> ← 1	2	1	3	8	3	5	1
<i>T</i> [2] ← <i>T</i> [1]	2	1	3	8	8	5	1
<i>j</i> ← 0	2	0	3	8	8	5	1
<i>T</i> [1] ← <i>x</i>	2	0	3	3	8	5	1
<i>i</i> ← 3	3	-	-	3	8	5	1
<i>x</i> ← <i>T</i> [3]	3	-	5	3	8	5	1
<i>j</i> ← 2	3	2	5	3	8	5	1
<i>T</i> [3] ← <i>T</i> [2]	3	2	5	3	8	8	1
<i>j</i> ← 1	3	1	5	3	8	8	1
<i>T</i> [2] ← <i>x</i>	3	1	5	3	5	8	1
<i>i</i> ← 4	4	-	-	3	5	8	1
<i>x</i> ← <i>T</i> [4]	4	-	1	3	5	8	1
<i>j</i> ← 3	4	3	1	3	5	8	1
<i>T</i> [4] ← <i>T</i> [3]	4	3	1	3	5	8	8
<i>j</i> ← 2	4	2	1	3	5	8	8
<i>T</i> [3] ← <i>T</i> [2]	4	2	1	3	5	5	8
<i>j</i> ← 1	4	1	1	3	5	5	8
<i>T</i> [2] ← <i>T</i> [1]	4	1	1	3	3	5	8
<i>j</i> ← 0	4	0	1	3	3	5	8
<i>T</i> [1] ← <i>x</i>	4	0	1	1	3	5	8

Lecture : Les valeurs en gras indiquent les cases modifiées à cette ligne. On voit clairement la zone triée s'agrandir à gauche (colonnes *T*[1]..*T*[*i* - 1] stables après chaque étape *i*).

5.6 Exécution détaillée sur un exemple (récapitulatif visuel)

Appliquons l'algorithme au tableau $T = [8, 3, 5, 1]$ de $N = 4$ éléments.

EXEMPLE Trace complète du tri de [8, 3, 5, 1]

Étape $i = 2$: $x = T[2] = 3$, $j = 1$. Comme $T[1] = 8 > 3$, on décale : $T[2] \leftarrow 8$, $j = 0$. La boucle

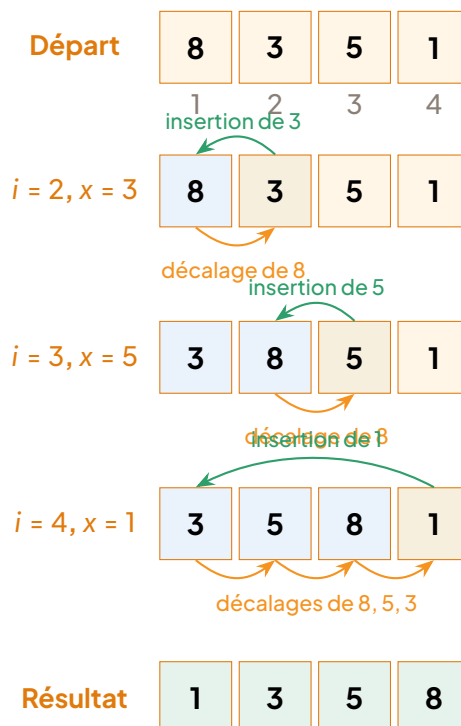
s'arrête ($j < 1$). On insère : $T[1] \leftarrow 3$. État : $[3, 8, 5, 1]$.

Étape $i = 3$: $x = T[3] = 5, j = 2$. $T[2] = 8 > 5$: décalage $T[3] \leftarrow 8, j = 1$. $T[1] = 3 \leq 5$: arrêt. On insère : $T[2] \leftarrow 5$. État : $[3, 5, 8, 1]$.

Étape $i = 4$: $x = T[4] = 1, j = 3$. $T[3] = 8 > 1$: $T[4] \leftarrow 8, j = 2$. $T[2] = 5 > 1$: $T[3] \leftarrow 5, j = 1$. $T[1] = 3 > 1$: $T[2] \leftarrow 3, j = 0$. Arrêt ($j < 1$). On insère : $T[1] \leftarrow 1$. État : $[1, 3, 5, 8]$.

Le tableau est trié. Récapitulatif synthétique :

Étape i	x	Décalages	Tableau après insertion
(départ)	—	—	$[8, 3, 5, 1]$
2	3	8	$[3, 8, 5, 1]$
3	5	8	$[3, 5, 8, 1]$
4	1	8, 5, 3	$[1, 3, 5, 8]$



Étapes du tri par insertion sur $[8, 3, 5, 1]$. En bleu : partie déjà triée; en doré : valeur x à insérer; flèches orange : décalages; flèche verte : insertion.

EXERCICE RÉSOLU 5 Trier les moyennes de quatre élèves

Énoncé. Les moyennes de quatre élèves sont stockées dans le tableau $M = [12, 7, 15, 9]$. Exécuter pas à pas l'algorithme TriInsertion et donner l'état du tableau après chaque

étape.

SOLUTION

Solution détaillée. On applique la boucle externe pour $i = 2, 3, 4$.

Étape $i = 2$: $x = M[2] = 7, j = 1$. $M[1] = 12 > 7$: décalage $M[2] \leftarrow 12, j = 0$; arrêt. Insertion $M[1] \leftarrow 7$. État : $[7, 12, 15, 9]$.

Étape $i = 3$: $x = M[3] = 15, j = 2$. $M[2] = 12 \leq 15$: la boucle interne ne s'exécute pas du tout. Insertion $M[3] \leftarrow 15$ (il reste en place). État : $[7, 12, 15, 9]$.

Étape $i = 4$: $x = M[4] = 9, j = 3$. $M[3] = 15 > 9$: $M[4] \leftarrow 15, j = 2$. $M[2] = 12 > 9$: $M[3] \leftarrow 12, j = 1$. $M[1] = 7 \leq 9$: arrêt. Insertion $M[2] \leftarrow 9$. État : $[7, 9, 12, 15]$.

Le tableau final trié par ordre croissant est $[7, 9, 12, 15]$. On remarque qu'à l'étape $i = 3$, aucun décalage n'a été nécessaire : l'élément était déjà à sa place.

5.7 Coût de l'algorithme et propriétés

Combien de comparaisons le tri par insertion effectue-t-il ? Analysons les cas extrêmes de manière intuitive.

- **Meilleur cas** : le tableau est déjà trié. À chaque étape i , la condition $T[j] > x$ est fausse dès le premier test : une seule comparaison par étape, soit environ N comparaisons au total. Aucun décalage n'est effectué.
- **Pire cas** : le tableau est trié dans l'ordre décroissant (par exemple $[9, 7, 5, 3]$). À l'étape i , l'élément x doit remonter jusqu'en première position : il faut $i - 1$ comparaisons et $i - 1$ décalages. Le nombre total de comparaisons est alors :

$$1 + 2 + 3 + \dots + (N - 1) = \frac{N(N - 1)}{2}$$

PROPRIÉTÉ Coût du tri par insertion (admis)

Le tri par insertion effectue, dans le pire des cas, de l'ordre de N^2 comparaisons (on dit qu'il est **quadratique**) : précisément $\frac{N(N - 1)}{2}$ comparaisons lorsque le tableau est trié en ordre inverse. Pour $N = 1000$, cela représente environ 500 000 comparaisons. En revanche, sur un tableau déjà trié ou presque trié, il n'effectue que de l'ordre de N comparaisons.

PROPRIÉTÉ Stabilité

Le tri par insertion est un tri **stable** : si deux éléments ont la même valeur, leur ordre relatif est préservé. En effet, la condition de décalage est $T[j] > x$ (strictement supérieur) : les

éléments égaux à x ne sont pas décalés, x s'insère après eux.

LESAVAIS-TU ? Le champion des tableaux presque triés

Le tri par insertion possède une qualité rare : plus le tableau est proche de l'ordre final, plus il est rapide. Sur un tableau « presque trié » (quelques éléments mal placés seulement), il bat des algorithmes réputés plus sophistiqués. C'est pourquoi les tris utilisés en pratique dans les bibliothèques des langages modernes (comme `std::sort` en C++ ou `Timsort` en Python/Java) combinent souvent une méthode puissante pour les grandes masses de données et un tri par insertion pour finir le travail sur les petites zones. C'est aussi le tri qu'utilisent naturellement les joueurs de cartes du monde entier, de Yaoundé à Tokyo, sans même savoir qu'ils appliquent un algorithme étudié en Terminale !

À RETENIR L'essentiel du tri par insertion

- Trier un tableau, c'est ordonner ses éléments (croissant ou décroissant). Pour l'ordre décroissant, il suffit de remplacer $T[j] > x$ par $T[j] < x$ dans la boucle interne.
- Le tri par insertion reproduit la méthode du joueur de cartes : à l'étape i , la partie $T[1..i-1]$ est triée et on insère $T[i]$ à sa place par décalages successifs.
- Trois variables clés : i (frontière), x (sauvegarde de $T[i]$), j (curseur de décalage).
- Ne jamais écraser $T[i]$ sans l'avoir sauvegardé dans x , et toujours tester $j \geq 1$ avant $T[j] > x$ pour éviter un débordement d'indice.
- Coût : de l'ordre de N^2 comparaisons dans le pire cas, mais linéaire (N) dans le meilleur cas et sur les tableaux presque triés.
- C'est un tri stable (préserve l'ordre des égaux) et en place (pas de tableau auxiliaire nécessaire).
- La table d'exécution est l'outil de validation rigoureuse : elle suit pas à pas les variables i, j, x et le contenu du tableau.

EXERCICE 3 Entraînement : Tri par insertion manuel

Donner l'état du tableau $T = [5, 2, 4, 6, 1, 3]$ après chaque itération de la boucle externe ($i = 2$ à 6) de l'algorithme `TriInsertion`. Construire la table d'exécution complète pour les deux premières itérations ($i = 2$ et $i = 3$ seulement).

CORRIGÉ Exercice n

États successifs du tableau :

- Init : [5, 2, 4, 6, 1, 3]
- $i = 2$ ($x = 2$) : décalage de 5 $\rightarrow$ [2, 5, 4, 6, 1, 3]
- $i = 3$ ($x = 4$) : décalage de 5 $\rightarrow$ [2, 4, 5, 6, 1, 3]
- $i = 4$ ($x = 6$) : déjà en place $\rightarrow$ [2, 4, 5, 6, 1, 3]
- $i = 5$ ($x = 1$) : décalages de 6, 5, 4, 2 $\rightarrow$ [1, 2, 4, 5, 6, 3]
- $i = 6$ ($x = 3$) : décalages de 6, 5, 4 $\rightarrow$ [1, 2, 3, 4, 5, 6]

Table d'exécution pour $i = 2$ et $i = 3$:

Instr.	i	j	x	$T[1]$	$T[2]$	$T[3]$	$T[4..6]$
Init	-	-	-	5	2	4	6, 1, 3
$i = 2$	2	-	-	5	2	4	6, 1, 3
$x = T[2]$	2	-	2	5	2	4	6, 1, 3
$j = 1$	2	1	2	5	2	4	6, 1, 3
$T[2] = T[1]$	2	1	2	5	5	4	6, 1, 3
$j = 0$	2	0	2	5	5	4	6, 1, 3
$T[1] = x$	2	0	2	2	5	4	6, 1, 3
$i = 3$	3	-	-	2	5	4	6, 1, 3
$x = T[3]$	3	-	4	2	5	4	6, 1, 3
$j = 2$	3	2	4	2	5	4	6, 1, 3
$T[3] = T[2]$	3	2	4	2	5	5	6, 1, 3
$j = 1$	3	1	4	2	5	5	6, 1, 3
$T[2] = x$	3	1	4	2	4	5	6, 1, 3

6 EXÉCUTION PAS À PAS ET TABLE D'EXÉCUTION

Nous venons d'écrire deux algorithmes fondamentaux : la **recherche séquentielle** et le **tri par insertion**. Une question naturelle se pose alors : comment être certain qu'un algorithme est correct avant de le traduire en langage C et de l'exécuter sur une machine ? Un programme qui boucle indéfiniment ou qui affiche un mauvais résultat coûte du temps, et l'erreur est souvent beaucoup plus facile à repérer sur le papier que dans un code compilé. La réponse des informaticiens est une technique simple et puissante : jouer soi-même le rôle de l'ordinateur, instruction par instruction, en notant tout ce qui se passe. C'est l'**exécution pas à pas**, matérialisée par la **table d'exécution**.

6.1 Pourquoi exécuter un algorithme à la main ?

DÉFINITION Exécution pas à pas

L'**exécution pas à pas** (ou trace d'exécution) d'un algorithme consiste à simuler mentalement le travail du processeur : on exécute les instructions une par une, dans l'ordre où la machine les rencontrerait, et l'on note après chaque instruction la valeur de chaque variable ainsi que ce qui s'affiche à l'écran.

Cette démarche répond à trois besoins concrets :

- **Vérifier** : s'assurer que l'algorithme produit bien le résultat attendu sur un exemple précis ;
- **Déboguer** : localiser l'instruction exacte où les valeurs deviennent fausses (boucle infinie, indice hors tableau, mauvaise initialisation) ;
- **Comprendre** : intérioriser le fonctionnement d'un algorithme en le « vivant » de l'intérieur, ce qui est indispensable pour l'épreuve d'informatique du Baccalauréat.

LESAVAIS-TU ? Le « test à sec » des grands programmeurs

En anglais, cette technique s'appelle le dry run (littéralement « essai à sec »). Avant l'existence des débogueurs intégrés, les pionniers de l'informatique comme Alan Turing traçaient leurs programmes à la main sur des feuilles quadrillées. Encore aujourd'hui, dans les concours et les entretiens d'embauche des grandes entreprises du numérique, on demande aux candidats d'exécuter un algorithme sur papier : c'est la preuve qu'on le comprend vraiment.

6.2 Structure d'une table d'exécution

DÉFINITION Table d'exécution

Une **table d'exécution** est un tableau à quatre colonnes :

1. Ligne : le numéro de la ligne de l'algorithme en cours d'exécution ;
2. Instruction : le texte de l'instruction exécutée (ou l'évaluation de la condition pour un test) ;
3. Variables : les valeurs de l'ensemble des variables après l'exécution de l'instruction (on ne reporte en pratique que celles qui changent, les autres conservant leur valeur) ;
4. Écran : ce qui s'affiche à l'écran (uniquement pour les instructions d'écriture).

Chaque ligne de la table correspond à une seule instruction exécutée.

MÉTHODE Construire une table d'exécution en quatre étapes

1. Numéroté les lignes de l'algorithme (chaque instruction simple reçoit un numéro ; les mots-clés de structure `FinSi`, `FinTantQue`, `FinPour` ne sont pas numérotés car ils ne s'exécutent pas) ;
2. Préparer le tableau : une colonne par variable susceptible de changer, plus les colonnes « ligne », « instruction » et « écran » ;
3. Exécuter les instructions dans l'ordre : à chaque instruction, écrire une nouvelle ligne de la table et mettre à jour uniquement la variable modifiée (les autres gardent leur valeur précédente) ;
4. Conclure : vérifier que le résultat final (valeurs des variables, affichages) correspond au résultat attendu.

ATTENTION L'erreur classique de la table d'exécution

L'erreur la plus fréquente consiste à recopier ou modifier toutes les variables à chaque ligne, comme si chaque instruction agissait sur tout. Or une instruction ne modifie qu'une seule variable (celle qui reçoit l'affectation) ; toutes les autres conservent leur valeur. Autre piège : oublier qu'un test (`Si`, `Tant que`) ne modifie aucune variable — il détermine seulement la prochaine instruction exécutée. Enfin, pour une boucle, il faut écrire autant de lignes que de passages dans la boucle, et non une seule ligne résumée.

6.3 Trace de la recherche séquentielle

Reprenons l'algorithme de recherche séquentielle (avec drapeau) vu à la section 4, en numérotant ses lignes. On cherche la valeur 17 dans le tableau $T = [5, 12, 17, 9]$ de $n = 4$ éléments (indices 1 à 4).

Algorithme RechercheSequentielle

Variables : T : tableau[1..4] d'entiers

i, n, x : entiers

trouve : booléen

Debut

```

1  n ← 4 ; x ← 17
2  trouve ← FAUX
3  i ← 1
4  Tant que (i ≤ n) et (non trouve) Faire
5      Si T[i] = x Alors
6          trouve ← VRAI
7      Sinon
8          i ← i + 1
9      FinSi
10 FinTantQue
11 Si trouve Alors
12     Ecrire("Valeur trouvée à la position ", i)
```

```

13  Sinon
14      Ecrire("Valeur absente")
15  FinSi
Fin

```

La figure suivante présente la table d'exécution complète. On y lit que la boucle s'arrête dès que `trouve` devient VRAI, c'est-à-dire à la position $i = 3$.

Ligne	Instruction	i	trouve	Écran
1	$n \leftarrow 4; x \leftarrow 17$	—	—	
2	$\text{trouve} \leftarrow \text{FAUX}$	—	FAUX	
3	$i \leftarrow 1$	1	FAUX	
4	test : $(1 \leq 4)$ et (non FAUX) $\rightarrow$ VRAI	1	FAUX	
5	test : $T[1] = 5 = 17?$ $\rightarrow$ FAUX	1	FAUX	
8	$i \leftarrow i + 1$	2	FAUX	
4	test : $(2 \leq 4)$ et (non FAUX) $\rightarrow$ VRAI	2	FAUX	
5	test : $T[2] = 12 = 17?$ $\rightarrow$ FAUX	2	FAUX	
8	$i \leftarrow i + 1$	3	FAUX	
4	test : $(3 \leq 4)$ et (non FAUX) $\rightarrow$ VRAI	3	FAUX	
5	test : $T[3] = 17 = 17?$ $\rightarrow$ VRAI	3	FAUX	
6	$\text{trouve} \leftarrow \text{VRAI}$	3	VRAI	
4	test : non VRAI $\rightarrow$ FAUX, sortie boucle	3	VRAI	
11	test : $\text{trouve} = \text{VRAI}$	3	VRAI	
12	Ecrire(...)	3	VRAI	Valeur trouvée à la position 3

Table d'exécution de la recherche de 17 dans [5,12,17,9]. Chaque ligne correspond à une instruction exécutée; seules les variables modifiées changent de valeur.

À RETENIR Ce que révèle la trace

La trace montre deux choses essentielles : (1) le drapeau `trouve` permet de sortir dès que la valeur est rencontrée, sans parcourir inutilement la fin du tableau ; (2) la variable i contient, à la sortie de la boucle, la position de la valeur cherchée. Si la valeur est absente, la boucle se termine quand i dépasse n (ici $i = 5$), et `trouve` reste FAUX.

6.4 Trace du tri par insertion

Traçons maintenant le tri par insertion (section 5) sur le tableau $T = [8, 3, 5, 1]$ (indices 1 à 4). Rappelons le principe : à l'étape i , on insère l'élément $T[i]$ (la **clé**) à sa place dans la partie déjà triée $T[1..i-1]$, en décalant vers la droite les éléments plus grands que lui.

Algorithme TriInsertion
Variables : T : tableau[1..4] d'entiers

```

    i, j, cle : entiers
Debut
1  Pour i allant de 2 a 4 Faire
2      cle ← T[i]
3      j ← i - 1
4      Tant que (j >= 1) et (T[j] > cle) Faire
5          T[j+1] ← T[j]      // decalage vers la droite
6          j ← j - 1
7      FinTantQue
8      T[j+1] ← cle           // insertion de la cle
9  FinPour
Fin

```

EXEMPLE Trace narrative du tri par insertion sur [8, 3, 5, 1]

Étape $i = 2$: $cle = T[2] = 3, j = 1$. Comme $T[1] = 8 > 3$, on décale : $T[2] \leftarrow 8, j = 0$. La boucle interne s'arrête ($j < 1$), puis $T[1] \leftarrow 3$. Tableau : [3, 8, 5, 1].

Étape $i = 3$: $cle = T[3] = 5, j = 2$. $T[2] = 8 > 5$: décalage, $T[3] \leftarrow 8, j = 1$. $T[1] = 3 \leq 5$: arrêt. $T[2] \leftarrow 5$. Tableau : [3, 5, 8, 1].

Étape $i = 4$: $cle = T[4] = 1, j = 3$. $T[3] = 8 > 1$: $T[4] \leftarrow 8, j = 2$. $T[2] = 5 > 1$: $T[3] \leftarrow 5, j = 1$. $T[1] = 3 > 1$: $T[2] \leftarrow 3, j = 0$. Arrêt, puis $T[1] \leftarrow 1$. Tableau final : [1, 3, 5, 8]. Le tableau est trié.

EXERCICE RÉSOLU 6 Table d'exécution du tri par insertion sur trois éléments

Énoncé : construire la table d'exécution complète du tri par insertion appliqué au tableau $T = [9, 4, 7]$ ($n = 3$). On notera l'état du tableau dans la colonne « variables ».

SOLUTION

Solution détaillée. On exécute l'algorithme ligne par ligne :

Ligne	Instruction	i	j	Tableau T / clé
1	$i \leftarrow 2$ (entrée boucle)	2	—	[9, 4, 7]
2	$cle \leftarrow T[2]$	2	—	$cle = 4$
3	$j \leftarrow 1$	2	1	$cle = 4$
4	test : $(1 \geq 1)$ et $(T[1] = 9 > 4) \rightarrow$ VRAI	2	1	$cle = 4$
5	$T[2] \leftarrow T[1]$	2	1	[9, 9, 7]
6	$j \leftarrow 0$	2	0	[9, 9, 7]
4	test : $(0 \geq 1) ?$ FAUX $\rightarrow$ sortie	2	0	[9, 9, 7]
8	$T[1] \leftarrow cle$	2	0	[4, 9, 7]
1	$i \leftarrow 3$ (itération suivante)	3	0	[4, 9, 7]
2	$cle \leftarrow T[3]$	3	0	$cle = 7$
3	$j \leftarrow 2$	3	2	$cle = 7$
4	test : $(2 \geq 1)$ et $(T[2] = 9 > 7) \rightarrow$ VRAI	3	2	$cle = 7$
5	$T[3] \leftarrow T[2]$	3	2	[4, 9, 9]
6	$j \leftarrow 1$	3	1	[4, 9, 9]
4	test : $(1 \geq 1)$ et $(T[1] = 4 > 7) ?$ FAUX $\rightarrow$ sortie	3	1	[4, 9, 9]
8	$T[2] \leftarrow cle$	3	1	[4, 7, 9]
1	fin de la boucle Pour ($i = 4 > 3$)	—	—	[4, 7, 9]

Le tableau final [4, 7, 9] est bien trié par ordre croissant : l'algorithme est validé sur ce jeu de test. Remarque le passage intermédiaire [9, 9, 7] : la valeur 4 n'est pas perdue, elle est sauvegardée dans `cle` avant le décalage. C'est précisément le rôle de cette variable.

6.5 Utiliser la trace pour détecter une erreur

La table d'exécution est un formidable outil de **débogage**. Deux erreurs classiques y apparaissent immédiatement.

1. La boucle infinie. Supposons qu'on ait oublié la ligne 8 ($i \leftarrow i + 1$) dans la recherche séquentielle. La trace le montre sans pitié : i reste bloqué à 1, le test de la ligne 4 est toujours VRAI, et la table s'allonge indéfiniment avec des lignes identiques. Dès qu'on remarque que plusieurs lignes consécutives sont strictement identiques, on sait que la boucle ne progressera jamais : il manque une instruction de progression (incrément).

2. La mauvaise initialisation. Supposons qu'on ait écrit `trouve  $\leftarrow$  VRAI` à la ligne 2 par erreur. La trace donne : à la ligne 4, le test « non trouve » est FAUX dès le départ, la boucle n'est jamais exécutée, et l'algorithme affiche « Valeur trouvée à la position 1 » alors que $T[1] = 5 \neq 17$. Le résultat final, absurde, révèle l'erreur d'initialisation.

ATTENTION Vérifier la condition d'arrêt

Toute boucle `Tant que` doit contenir, dans son corps, au moins une instruction qui fait évoluer une variable de la condition d'arrêt. Avant de tracer, pose-toi la question : «

qu'est-ce qui rapproche la boucle de sa fin ? ». Si la réponse n'est pas évidente, la trace le confirmera : c'est le premier symptôme d'une boucle infinie.

6.6 Valider un algorithme : le choix des jeux de tests

Une seule trace réussie ne prouve pas qu'un algorithme est correct : elle prouve seulement qu'il fonctionne sur cet exemple. Pour **valider** sérieusement un algorithme, on le teste sur plusieurs **jeux de tests** choisis avec soin.

MÉTHODE Choisir ses jeux de tests

Pour valider un algorithme de recherche ou de tri, prévois au minimum :

1. Le cas normal : une valeur présente au milieu du tableau, un tableau quelconque non trié (ex. [8, 3, 5, 1]) ;
2. Les cas limites : valeur en première position, valeur en dernière position, tableau à un seul élément, tableau déjà trié [1, 3, 5, 8], tableau trié en ordre inverse [8, 5, 3, 1] (c'est le cas le plus coûteux pour le tri par insertion) ;
3. Le cas impossible : valeur absente du tableau (la recherche doit répondre « absente » et non boucler ou planter), tableau vide si le problème l'autorise.

L'algorithme n'est validé que s'il donne le bon résultat sur tous ces cas.

EXEMPLE Jeux de tests pour la recherche séquentielle sur $T = [5, 12, 17, 9]$

Type de cas	Valeur x	Résultat attendu
Normal	12	trouvée, position 2
Limite (début)	5	trouvée, position 1
Limite (fin)	9	trouvée, position 4
Impossible	100	« Valeur absente »

Si l'algorithme échoue sur l'un de ces cas (par exemple une boucle qui s'arrête avant la dernière case à cause d'un test $i < n$ au lieu de $i \leq n$), le cas limite « fin » le démasquera.

EXPÉRIENCE TP : valider le tri par insertion à la main puis en C

Objectif : confronter la trace papier à l'exécution réelle.

Matériel : papier quadrillé, crayon, puis un ordinateur avec un compilateur C (Code ::Blocks ou GCC).

Manipulation 1 (papier) : trace le tri par insertion sur les quatre jeux de tests suivants : [8, 3, 5, 1] (cas normal), [1, 2, 3, 4] (déjà trié), [4, 3, 2, 1] (inversé), [7] (un seul élément). Compte pour chacun le nombre de décalages (ligne 5 exécutée).

Manipulation 2 (machine) : saisis et exécute le programme C suivant, qui affiche le tableau trié et le nombre total de décalages.

```
#include <stdio.h>

int main(void) {
    int T[4] = {8, 3, 5, 1}; /* jeu de test à modifier */
    int n = 4, i, j, cle, decalages = 0;

    for (i = 1; i < n; i++) { /* indices C : 0 à n-1 */
        cle = T[i];
        j = i - 1;
        while (j >= 0 && T[j] > cle) {
            T[j + 1] = T[j]; /* décalage vers la droite */
            j = j - 1;
            decalages = decalages + 1;
        }
        T[j + 1] = cle; /* insertion de la cle */
    }

    for (i = 0; i < n; i++) {
        printf("%d", T[i]);
    }
    printf("\nDecalages : %d\n", decalages);
    return 0;
}
```

Observations : pour [8, 3, 5, 1], le programme affiche 1 3 5 8 et 5 décalages — exactement ce que la trace papier avait donnée. Pour le tableau déjà trié, 0 décalage ; pour le tableau inversé, 6 décalages (le maximum).

Interprétation : la concordance entre la trace manuelle et l'exécution réelle valide l'algorithme. Le tri par insertion est très rapide sur un tableau presque trié et le plus lent sur un tableau inversé : on parle de meilleur et de pire des cas.

EXERCICE 4 À toi de jouer

1. Construis la table d'exécution de la recherche séquentielle de la valeur 100 dans $T = [5, 12, 17, 9]$. Que vaut i à la sortie de la boucle ?
2. Trace le tri par insertion sur $T = [2, 1]$ puis sur $T = [1, 2]$. Compare le nombre d'instructions exécutées.
3. Un élève a écrit, dans la boucle du tri par insertion, le test Tant que $(j \geq 1)$ et $(T[j] \geq cle)$ (avec $\geq$ au lieu de $>$). Trace cet algorithme sur [5, 5, 3]. Le résultat est-il encore

correct? Que peut-on dire du nombre de décalages et de la **stabilité** du tri?

CORRIGÉ Exercice

1. La boucle s'exécute quatre fois sans jamais trouver 100 ; i prend successivement les valeurs 1, 2, 3, 4, 5. À la sortie, $i = 5 > n = 4$ et `trouve = FAUX` : l'algorithme affiche « Valeur absente ». C'est le comportement attendu.
2. Pour [2, 1] : un décalage ($T[2] \leftarrow 2$) puis $T[1] \leftarrow 1$; résultat [1, 2]. Pour [1, 2] : le test $T[1] > 2$ est FAUX immédiatement, aucun décalage. Le tableau déjà trié coûte beaucoup moins cher (meilleur cas).
3. Le tableau [5, 5, 3] est trié correctement en [3, 5, 5] : le résultat reste juste. Mais avec $\geq$, les éléments égaux à la clé sont aussi décalés, ce qui provoque des décalages inutiles (le tri perd son caractère stable — l'ordre relatif des égaux n'est plus conservé — et devient légèrement plus lent). La condition correcte pour un tri stable et optimal est bien $T[j] > cle$.

À RETENIR L'essentiel de la section

- La **table d'exécution** simule l'ordinateur : une ligne par instruction, colonnes ligne / instruction / variables / écran.
- Une instruction ne modifie qu'une variable ; un test n'en modifie aucune.
- La trace permet de vérifier un résultat, de détecter les boucles infinies et les mauvaises initialisations.
- Un algorithme n'est validé qu'après plusieurs **jeux de tests** : cas normal, cas limites (début, fin, déjà trié, inversé, un élément) et cas impossible (valeur absente).

7 ACTIVITÉ D'INTÉGRATION

8 LEÇON 07 : ÉCRITURE ET EXÉCUTION D'ALGORITHMES INTÉGRANT UNE STRUCTURE DE DONNÉES

8.1 Activité d'intégration : Le Tableau d'Honneur du Lycée Bilingue de Nkolbisson

8.2 Objectifs de l'activité

Cette activité de synthèse mobilise l'ensemble des compétences de la leçon dans une situation authentique. À l'issue de ce travail, vous devez être capable de :

- Concevoir une structure de données adaptée (**tableau d'enregistrements**) pour modéliser une liste d'élèves avec leurs moyennes.
- Implémenter une **recherche séquentielle** avec drapeau sur un champ précis d'un tableau d'enregistrements.
- Adapter l'algorithme du **tri par insertion** pour trier des structures complexes selon un critère (la moyenne) dans l'ordre décroissant.
- Réaliser une **table d'exécution** rigoureuse pour valider le comportement de vos algorithmes (recherche et tri).
- Présenter et défendre vos résultats dans une démarche de validation croisée, propre à la pratique professionnelle du développement logiciel.

8.3 Situation

Le Proviseur du **Lycée Bilingue de Nkolbisson** (Yaoundé) souhaite publier le Tableau d'Honneur du premier trimestre pour la classe de Terminale C. Il remet au professeur principal une liste des moyennes des 8 élèves de la classe, saisies dans le désordre par le secrétariat :

{11, 15, 8, 13, 17, 10, 14, 12}

Chaque moyenne est associée à un élève (nom + prénom), mais le fichier transmis ne contient que les notes. Pour les besoins de l'activité, on supposera que les noms correspondent à l'ordre de saisie initial: ["Abena", "Biya", "Chou", "Dongmo", "Essomba", "Fouda", "Gueye", "Haman"].

Votre groupe, en tant que brigade informatique de la classe, est chargé de :

1. Structurer ces données proprement.
2. Vérifier si un élève a obtenu exactement **14/20** (seuil symbolique pour une mention "Assez Bien" au Bac) et identifier sa position dans la liste initiale.
3. Classer l'ensemble des élèves par moyenne **décroissante** pour établir le palmarès officiel.
4. Extraire le **Top 3** pour l'affichage au tableau d'honneur.

Le Proviseur exige une traçabilité totale : chaque algorithme (recherche et tri) doit être accompagné de sa table d'exécution complète, présentée au tableau pour validation par les autres groupes.

8.4 Consignes

Travail en groupes de 3 à 4 élèves. Durée : 2 heures. Chaque groupe rend une fiche unique contenant les réponses aux 4 questions ci-dessous.

1. Modélisation des données

Écrire la déclaration complète (dans le formalisme algorithmique du cours) :

- Du type **Enregistrement** Eleve (champs : Nom : Chaîne, Moyenne : Réel).
- Du tableau Promo de 8 éléments de type Eleve.
- De l'initialisation du tableau Promo avec les 8 couples (Nom, Moyenne) donnés dans l'énoncé.

2. Recherche séquentielle avec table d'exécution

Écrire l'algorithme RechercheNote14 qui parcourt Promo pour trouver le **premier** élève dont la moyenne vaut 14.

- Utiliser un **drapeau** (booléen Trouve) et un indice Pos.
- Afficher : "Élève trouvé : [Nom] à la position [Pos]" ou "Aucun élève avec 14".
- Construire la **table d'exécution complète** (colonnes : N° Ligne, Instruction, Variables modifiées, Écran) pour cet algorithme sur les données réelles. Combien de tours de boucle sont effectués ?

3. Tri par insertion décroissant avec trace

Adapter l'algorithme de tri par insertion vu en cours (qui trie par ordre croissant) pour trier le tableau Promo par **moyenne décroissante** (la plus grande en premier).

- Écrire l'algorithme TriDecroissant (modification de la condition de comparaison dans le TantQue).
- Réaliser la **trace complète pas à pas** (table d'exécution ou tableau d'états du tableau après chaque itération de la boucle principale Pour i de 1 à 7). Montrer clairement les décalages et l'insertion de la clé (Cle).

4. Affichage du Tableau d'Honneur

Écrire l'algorithme AfficherTop3 qui affiche les 3 premiers éléments du tableau trié sous la forme :

```
=== TABLEAU D'HONNEUR TLE C ===
1. Essomba : 17.00
2. Biya   : 15.00
3. Gueye  : 14.00
```

5. Validation croisée

Chaque groupe affiche sa table d'exécution du tri (Question 3) au tableau. Les autres groupes vérifient la cohérence des décalages et de l'état final. Toute différence doit être débattue et corrigée collectivement.

8.5 Ressources à mobiliser

EXEMPLE Syntaxe d'un enregistrement

```
Type Eleve = Enregistrement  
  Nom : Chaîne  
  Moyenne : Reel  
Fin Enregistrement
```

EXEMPLE Parcours de tableau – Convention 0-based

Pour $i \leftarrow 0$ à $N-1$ Faire ... FinPour (indices 0 à $N-1$). Taille $N = 8 \Rightarrow$ indices 0..7.

EXEMPLE Recherche séquentielle avec drapeau (rappel)

```
Trouve  $\leftarrow$  Faux ;  $i \leftarrow 0$   
TantQue ( $i < N$ ) ET (Non Trouve) Faire  
  Si  $\text{Tab}[i].\text{Champ} = \text{ValeurCherchee}$  Alors  
    Trouve  $\leftarrow$  Vrai ;  $\text{Pos} \leftarrow i$   
  Sinon  $i \leftarrow i + 1$   
FinSi  
FinTantQue
```

EXEMPLE Tri par insertion croissant (rappel)

```
Pour  $i \leftarrow 1$  à  $N-1$  Faire  
  Cle  $\leftarrow \text{Tab}[i]$  ;  $j \leftarrow i - 1$   
  TantQue ( $j >= 0$ ) ET ( $\text{Tab}[j] > \text{Cle}$ ) Faire  
     $\text{Tab}[j+1] \leftarrow \text{Tab}[j]$  ;  $j \leftarrow j - 1$   
  FinTantQue  
   $\text{Tab}[j+1] \leftarrow \text{Cle}$   
FinPour
```

PROPRIÉTÉ Adaptation pour ordre décroissant sur champ

Pour trier un tableau d'enregistrements par un champ numérique décroissant, on inverse la comparaison dans le TantQue : ($j >= 0$) ET ($\text{Tab}[j].\text{Champ} < \text{Cle}.\text{Champ}$). On décale tant que l'élément courant est plus petit que la clé (on veut les grands devant). L'affecta-

tion `Tab[j+1] <- Tab[j]` déplace la structure entière (tous les champs).

EXEMPLE Structure d'une table d'exécution

Colonnes obligatoires : N° Ligne | Instruction | Variables (Valeurs) | Écran. Une ligne par instruction exécutée.

8.6 Démarche et corrigé commenté

EXERCICE RÉSOLU 7

Question 1 : Modélisation des données Algorithme principal et déclaration des types

```
Algorithme TableauHonneur
// --- Types ---
Type Eleve = Enregistrement
    Nom : Chaîne
    Moyenne : Reel
Fin Enregistrement

// --- Variables ---
Promo : Tableau[0..7] de Eleve
i : Entier

// --- Initialisation des données (ordre de saisie initial) ---
Promo[0] <- {"Abena", 11} ; Promo[1] <- {"Biya", 15}
Promo[2] <- {"Chou", 8} ; Promo[3] <- {"Dongmo", 13}
Promo[4] <- {"Essomba", 17} ; Promo[5] <- {"Fouda", 10}
Promo[6] <- {"Gueye", 14} ; Promo[7] <- {"Haman", 12}

// Suite : Appels des modules Recherche, Tri, AffichageTop3
FinAlgorithme
```

ATTENTION Indices et initialisation

Respectez l'indice de départ 0 (convention du cours). L'initialisation "en dur" ligne par ligne est la traduction algorithmique d'un littéral de tableau. En langage C, on écrirait : `Eleve Promo[8] = {{"Abena",11}, {"Biya",15}, ...};`.

MÉTHODE Recherche séquentielle sur tableau d'enregistrements

1. Initialiser `Trouve` $\leftarrow$ Faux et `i` $\leftarrow$ 0.
2. Tant que `i` $\leq$ 7 ET Non `Trouve` faire :
3. Accéder au champ `Promo[i].Moyenne`.
4. Si égal à 14 : `Trouve` $\leftarrow$ Vrai, `Pos` $\leftarrow$ i.
5. Sinon : `i` $\leftarrow$ i + 1.
6. Après la boucle, tester `Trouve` pour l'affichage.

EXERCICE RÉSOLU 8**Question 2 : Recherche séquentielle de la note 14** Algorithme RechercheNote14

```
// Variables locales
Trouve : Booleen
Pos, i : Entier

Debut RechercheNote14
Trouve  $\leftarrow$  Faux
i  $\leftarrow$  0
TantQue (i  $\leq$  7) ET (Non Trouve) Faire
    Si Promo[i].Moyenne = 14 Alors
        Trouve  $\leftarrow$  Vrai
        Pos  $\leftarrow$  i
    Sinon
        i  $\leftarrow$  i + 1
    FinSi
FinTantQue

Si Trouve Alors
    Afficher("Élève trouvé : ", Promo[Pos].Nom, " à la position ", Pos)
Sinon
    Afficher("Aucun élève avec la moyenne 14")
FinSi
Fin RechercheNote14
```

Table d'exécution formelle (extrait significatif)

N° Ligne	Instruction exécutée	Variables modifiées (Valeurs)
1	Trouve $\leftarrow$ Faux	Trouve=Faux
2	i $\leftarrow$ 0	i=0
3	TantQue (0 $\leq$ 7) ET (Vrai)	-
4	Si Promo[0].Moyenne=14 (11=14 Faux)	-
5	i $\leftarrow$ 1	i=1
... itérations similaires pour i=1 (15), i=2 (8), i=3 (13), i=4 (17), i=5 (10) ...		
20	TantQue (6 $\leq$ 7) ET (Vrai)	-
21	Si Promo[6].Moyenne=14 (14=14 Vrai)	-
22	Trouve $\leftarrow$ Vrai	Trouve=Vrai
23	Pos $\leftarrow$ 6	Pos=6
24	FinTantQue (condition Non Trouve devient Faux)	-
25	Si Trouve Alors	-
26	Afficher(...)	-

Écran

-

-

-

-

-

-

-

-

-

-

-

-

Élève
trouvé :
Gueye
à
la
po-
si-
tion
6

À RETENIR Analyse de la recherche

La boucle TantQue a effectué 7 itérations (tests pour $i = 0$ à 6). L'instruction $i \leftarrow i+1$ a été exécutée 6 fois. L'algorithme s'arrête dès la première occurrence (indice 6). Complexité en temps : $O(N)$ dans le pire cas (8 tests si valeur absente ou en dernière position).

MÉTHODE Tri par insertion décroissant sur enregistrements

1. La clé Cle est de type Eleve (on déplace l'enregistrement entier).
2. Boucle principale : Pour i $\leftarrow$ 1 à 7 Faire ($N = 8$).
3. Cle $\leftarrow$ Promo[i] ; j $\leftarrow$ i-1.

4. **Condition de décalage** : TantQue ($j \geq 0$) ET ($\text{Promo}[j].\text{Moyenne} < \text{Cle}.\text{Moyenne}$) Faire.
5. $\text{Promo}[j+1] \leftarrow \text{Promo}[j]$ (décalage de la structure complète vers la droite).
6. $j \leftarrow j - 1$.
7. $\text{Promo}[j+1] \leftarrow \text{Cle}$ (insertion de la clé à sa place).

EXERCICE RÉSOLU 9**Question 3 : Tri par insertion décroissant Algorithme TriDecroissant**

```
// Variables locales
Cle : Eleve
i, j : Entier

Debut TriDecroissant
Pour i <- 1 à 7 Faire
    Cle <- Promo[i]    // Sauvegarde de l'élève à insérer
    j <- i - 1
    // Décalage des moyennes PLUS PETITES que Cle.Moyenne
    TantQue (j >= 0) ET (Promo[j].Moyenne < Cle.Moyenne) Faire
        Promo[j+1] <- Promo[j] // Décalage structure entière
        j <- j - 1
    FinTantQue
    Promo[j+1] <- Cle    // Insertion
FinPour
Fin TriDecroissant
```

Trace complète : États du tableau `Promo` après chaque itération `i` (Boucle Pour)

i	État Opérations clés (Boucle TantQue) du ta- bleau Promo (Moyenne - Nom)
Init	[11-Abena, 15-Biya, 8-Chou, 13-Dongmo, 17-Essomba, 10-Fouda, 14-Gueye, 12-Haman]
1	[15-Biya, 15-Biya). 15 > 11 : décalage Abena → ind 1. Insertion Biya ind 0. 11-Abena, 8-Chou, 13-Dongmo, 17-Essomba, 10-Fouda, 14-Gueye, 12-Haman]
2	[15-Biya, 8-Chou). 8 < 11 : pas de décalage. Insertion ind 2 (inchangé). 11-Abena, 8-Chou, 13-Dongmo, 17-Essomba, 10-Fouda, 14-Gueye, 12-Haman]
3	[15-Biya, 13-Dongmo). 13 > 8 (décalage Chou), 13 > 11 (décalage Abena), 13 < 15 stop. Insertion ind 1. 13-Dongmo, 11-Abena, 8-Chou, 17-Essomba, 10-Fouda, 14-Gueye, 12-Haman]
4	[17-Essomba, 17-Essomba). 17 > 8, > 11, > 13, > 15 : 4 décalages. Insertion ind 0. 15-Biya, 13-Dongmo, 11-Abena, 8-Chou, 10-Fouda, 14-Gueye, 12-Haman]
5	[17-Essomba, 10-Fouda). 10 > 8 (décalage Chou), 10 < 11 stop. Insertion ind 4. 15-Biya, 13-Dongmo, 11-Abena, 10-Fouda, 8-Chou, 14-Gueye, 12-Haman]
6	[17-Essomba, 14-Gueye). 14 > 8, > 10, > 11, > 13 (4 décalages), 14 < 15 stop. Insertion ind 2. 15-Biya, 14-Gueye,

ATTENTION Pièges fréquents sur le tri d'enregistrements

- Séparer les champs : N'affectez jamais seulement `Promo[j+1].Moyenne ← Promo[j].Moyenne`. Le nom resterait à l'ancienne place → incohérence fatale. Solution : Affectation structurelle `Promo[j+1] ← Promo[j]`.
- Sens de l'inégalité : Pour décroissant, on décale si `Tab[j].Champ < Cle.Champ`. Garder `>` (croissant) est l'erreur n°1.
- Ordre dans le TantQue : `(j >= 0) ET (Promo[j].Moyenne < Cle.Moyenne)`. La borne `j >= 0` doit être testée en premier (évaluation paresseuse) pour éviter un accès hors limites `Promo[-1]`.

EXERCICE RÉSOLU 10

Question 4 : Affichage du Top 3 Algorithme AfficherTop3

```

Debut AfficherTop3
Afficher("=== TABLEAU D'HONNEUR TLE C ===")
Pour i <- 0 à 2 Faire
    Afficher(i+1, ". ", Promo[i].Nom, " : ", Promo[i].Moyenne)
FinPour
Fin AfficherTop3

```

Sortie écran finale attendue :

```

=== TABLEAU D'HONNEUR TLE C ===
1. Essomba : 17
2. Biya : 15
3. Gueye : 14

```

8.7 Validation croisée (Bilan collectif)

Les groupes comparent leur trace du **Tri (Question 3)**. Points de contrôle critiques :

- Itération `i=4` (insertion Essomba/17) : le tableau devient-il `[17, 15, 13, 11, 8, ...]` ?
- Itération `i=6` (insertion Gueye/14) : vérifiez que 13 et 11 ont été décalés vers la droite, mais 15 est resté en place (arrêt du TantQue).
- État final (après `i=7`) : strictement décroissant 17, 15, 14, 13, 12, 11, 10, 8.

Toute divergence indique une erreur dans la condition du TantQue ou dans la gestion de l'indice `j`. Cette démarche simule la revue de code (code review) en entreprise.

8.8 Bilan de l'activité

Cette activité a permis de mettre en évidence plusieurs points fondamentaux de l'algorithmique appliquée :

1. **L'encapsulation des données** : Le passage du tableau simple (moyennes seules) au **tableau d'enregistrements** (`Promo`) est indispensable en situation réelle. On ne trie pas des notes "en l'air", on trie des dossiers élèves. L'erreur classique (séparer le nom de la note lors du tri) a été évitée grâce à l'affectation de structure globale (`Promo[j+1] ← Promo[j]`).
2. **La généricité des algorithmes classiques** : La recherche séquentielle et le tri par insertion s'appliquent identiquement sur des structures complexes, à condition de cibler le bon champ (`.Moyenne`) dans les comparaisons. L'adaptation "Croissant → Décroissant" ne change qu'un seul symbole (`>` devient `<`), mais change radicalement le résultat.
3. **La rigueur de la table d'exécution** : C'est l'outil unique pour **prouver** la correction d'un algorithme itératif. Sans elle, le tri par insertion (avec ses décalages imbriqués) est une "boîte noire" source de bugs. La validation croisée entre groupes simule la revue de code professionnelle.
4. **La complexité en action** : Sur $N = 8$, la recherche a pris 7 tests (pire cas : 8). Le tri par insertion a effectué $\sum_{i=1}^7 i = 28$ comparaisons dans le pire cas (tableau initialement croissant), mais moins ici car le désordre initial était aléatoire. Cela illustre concrètement la complexité quadratique $O(N^2)$.

LESAVAIS-TU ?

Le saviez-vous ?

Au Cameroun, la gestion des bulletins et tableaux d'honneur est de plus en plus informatisée via des logiciels comme GEESCO (Gestion Scolaire) ou des développements locaux (souvent en Python/Django ou PHP/MySQL aujourd'hui, mais la logique algorithmique reste celle apprise ici : structures, tris, recherches). Le Ministère des Secondaires Enseignements (MINESEC) encourage la numérisation des notes pour la transparence et la rapidité de publication des résultats du Baccalauréat. Maîtriser ces algorithmes, c'est comprendre le "moteur" de ces applications officielles.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 ÉCRITURE ET EXÉCUTION D'ALGORITHMES INTÉGRANT UNE STRUCTURE DE DONNÉES

10.1 Rappels : Structure d'un algorithme et instructions de base

DÉFINITION Algorithme

Une suite finie et non ambiguë d'instructions élémentaires permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée. En pseudo-code (convention Bac Cameroun), il suit la structure canonique :

1. **En-tête** : Algorithme <Nom>.
2. **Déclarations** : Constantes, Types, Variables (nom : type).
3. **Corps** : Début ... Fin.

PROPRIÉTÉ Instructions fondamentales

- **Affectation** : Variable $\leftarrow$ Expression (flèche, pas signe =).
- **Entrée/Sortie** : Lire(Variable), Écrire(Expression).
- **Conditionnelle** : Si <Cond> Alors <Inst> Sinon <Inst> FinSi (ou Si ... Alors ... FinSi).
- **Boucles bornées** : Pour Var de Debut à Fin [Pas] Faire ... FinPour.
- **Boucles non bornées** : TantQue <Cond> Faire ... FinTantQue (test en entrée); Répéter ... Jusqu'À <Cond> (test en sortie).

EXEMPLE Algorithme simple : Calcul de la moyenne de 3 notes

```
Algorithme MoyenneTroisNotes
Variables
  N1, N2, N3, Moy : Réel
Début
  Écrire("Entrez les 3 notes : ")
```

```
Lire(N1); Lire(N2); Lire(N3)
Moy <- (N1 + N2 + N3) / 3
Écrire("La moyenne est : ", Moy)
Fin
```

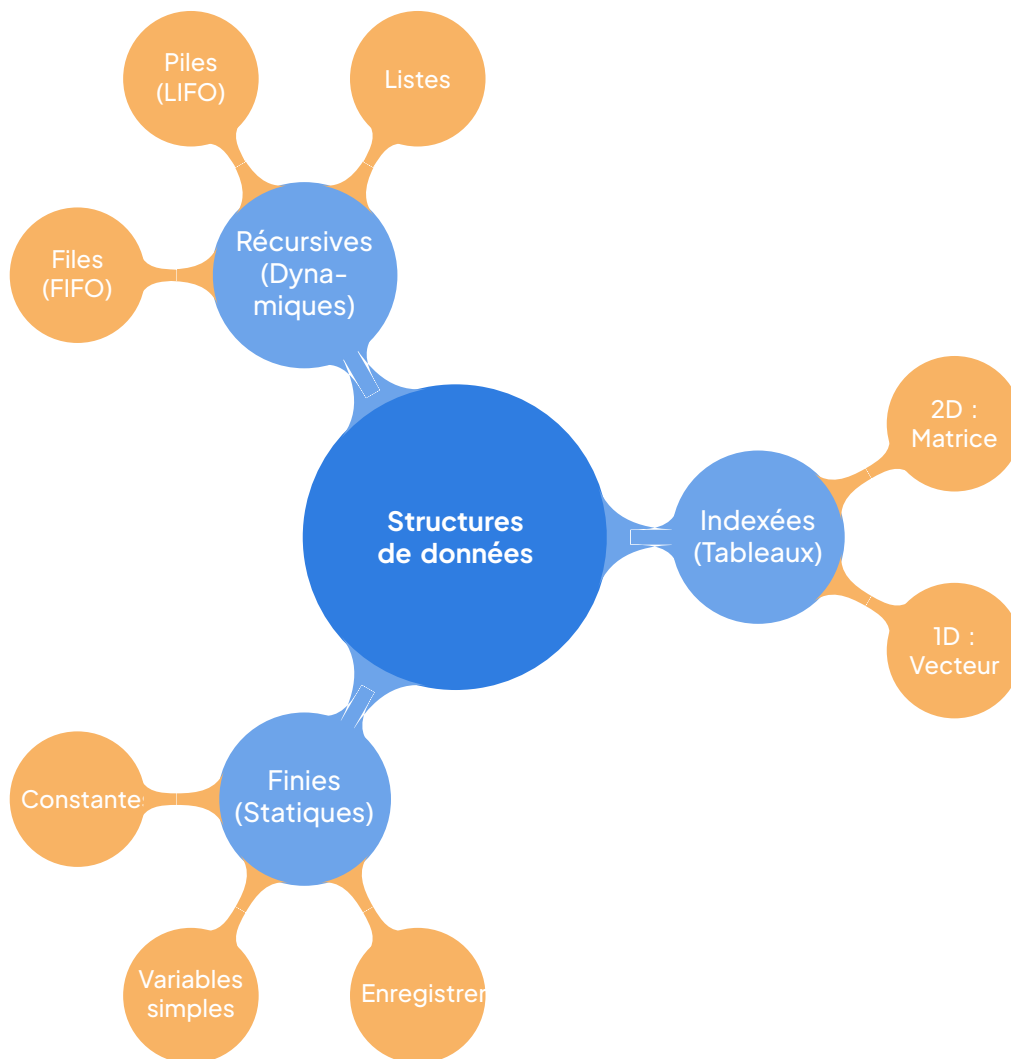
10.2 Les structures de données : classification

DÉFINITION Structure de donnée

Organisation logique des données en mémoire pour en faciliter l'accès, la modification et le traitement. Le programme de Terminale en distingue trois familles :

PROPRIÉTÉ Les trois familles de structures (Programme MINESEC)

- Structures finies (statiques) : Taille fixée à la compilation.
 - **Constante** : Valeur immuable (Constante $\pi = 3,14$).
 - **Variable simple** : Une seule valeur typée (Âge : Entier).
 - **Enregistrement (Structure hétérogène)** : Regroupe des champs nommés de types différents décrivant une même entité (ex : Eleve : Nom, Âge, Moyenne).
- Structures indexées (Tableaux) : Collection ordonnée d'éléments de même type, accessibles par un **indice** (entier). En pseudo-code Bac : indices de 1 à N.
- Structures récursives (Dynamiques) : Taille variable à l'exécution, gérées par pointeurs/références (Listes chaînées, Piles, Files). Étudiées en langage C (Module 3), conceptuellement connues en pseudo-code.



Classification des structures de données au programme de Terminale

10.3 Déclarer et parcourir un tableau unidimensionnel

MÉTHODE Déclaration, saisie et affichage d'un tableau

Objectif : Manipuler une collection ordonnée de données de même type (notes, prix, âges).

Étapes :

1. Identifier le besoin : Combien de valeurs ? Quel type (entier, réel, chaîne) ? → Taille N et Type T .
2. Déclarer le tableau : `Tableau Tab[1..N]` de Type (indices 1 à N inclus, convention Bac).
3. Saisir les éléments (Parcours en lecture) :

- Initialiser une boucle Pour i de 1 à N faire.
- Afficher un invite : Écrire("Élément n°", i , " : ").
- Lire : Lire(Tab[i]).

4. Afficher les éléments (Parcours en écriture) :

- Boucle Pour i de 1 à N faire.
- Écrire(Tab[i]) (ou Écrire("Tab[", i , "] = ", Tab[i]) pour déboguer).

Réflexe Bac : Toujours vérifier les bornes de la boucle (1 à N). Un indice 0 ou $N + 1$ provoque une erreur d'exécution (dépassement de capacité).

EXERCICE RÉSOLU 11

Gestion des notes d'une classe de Terminale C au Lycée de Nkolbisson

Énoncé : On souhaite saisir les notes de Mathématiques (sur 20) de 30 élèves, les stocker, puis les afficher avec leur rang. Écrire l'algorithme complet.

SOLUTION

Analyse : $N = 30$ (constant), Type = Entier (notes entières sur 20). On utilise un tableau Notes[1..30]. **Algorithme :**

```

Algorithme GestionNotes
Constante
    MAX_ELEVES = 30
Variables
    Notes : Tableau[1..MAX_ELEVES] de Entier
    i : Entier
Début
    // Saisie
    Pour i de 1 à MAX_ELEVES faire
        Écrire("Note de l'élève n°", i, " (sur 20) : ")
        Lire(Notes[i])
        // Optionnel : validation 0 <= Notes[i] <= 20
    FinPour

    // Affichage avec rang
    Écrire("--- LISTE DES NOTES ---")
    Pour i de 1 à MAX_ELEVES faire
        Écrire("Rang ", i, " : ", Notes[i], "/20")
    FinPour
Fin
  
```

Conclusion : L'algorithme structure les données dans un tableau, garantit l'ordre de saisie (rang = indice) et permet un traitement ultérieur (moyenne, max, recherche).

10.4 Manipuler un enregistrement (structure hétérogène)

MÉTHODE Déclaration et accès aux champs d'un enregistrement

Objectif : Regrouper des informations de types différents décrivant une même entité réelle (un élève, un livre, un employé).

Étapes :

1. **Modéliser l'entité :** Lister les attributs (Nom : Chaîne, Âge : Entier, Moyenne : Réel).
2. **Déclarer le type Enregistrement :** `Type Eleve = Enregistrement`
`Nom : Chaîne`
`Age : Entier`
`Moyenne : Réel`
`FinEnregistrement`
3. **Déclarer une variable de ce type :** `UnEleve : Eleve.`
4. **Accéder à un champ :** Notation pointée `Variable.Champ` (ex : `UnEleve.Nom`, `UnEleve.Moyenne`).
5. **Saisir/Afficher :** Traiter chaque champ indépendamment avec `Lire / Écrire`.

Attention : On ne peut pas écrire `Lire(UnEleve)` ni `Écrire(UnEleve)` globalement en pseudo-code standard Bac. Il faut traiter champ par champ.

EXERCICE RÉSOLU 12 Fiche élève pour le bulletin numérique MINEDUB

Énoncé : Définir un type `Eleve` contenant : Matricule (chaîne 10 car.), Nom (chaîne), Prénom (chaîne), Série (chaîne : "C", "D" ou "E"), MoyenneAnnuelle (réel). Écrire l'algorithme qui saisit les infos d'un élève et affiche son admission au Bac (Admis si Moyenne $\geq$ 10, Ajourné sinon).

SOLUTION

Algorithme :

```
Algorithme BulletinEleve
Type
  Eleve = Enregistrement
    Matricule : Chaîne
    Nom : Chaîne
    Prenom : Chaîne
    Serie : Chaîne
    MoyenneAnnuelle : Réel
  FinEnregistrement

Variables
  E : Eleve
```

```

Decision : Chaîne
Début
    Écrire("--- SAISIE FICHE ÉLÈVE ---")
    Écrire("Matricule : "); Lire(E.Matricule)
    Écrire("Nom : "); Lire(E.Nom)
    Écrire("Prénom : "); Lire(E.Prenom)
    Écrire("Série (C/D/E) : "); Lire(E.Serie)
    Écrire("Moyenne Annuelle : "); Lire(E.MoyenneAnnuelle)

    Si E.MoyenneAnnuelle >= 10 Alors
        Decision <- "ADMIS AU BACCALAURÉAT"
    Sinon
        Decision <- "AJOURNÉ"
    FinSi

    Écrire("")
    Écrire("=== BULLETIN OFFICIEL ===")
    Écrire("Matricule : ", E.Matricule)
    Écrire("Nom Complet : ", E.Prenom, " ", E.Nom)
    Écrire("Série : ", E.Serie)
    Écrire("Moyenne : ", E.MoyenneAnnuelle : 0 : 2, "/20")
    Écrire("Décision : ", Decision)
Fin

```

Justification : L'enregistrement structure le dossier élève. Le test sur E.MoyenneAnnuelle illustre l'accès champ. Le formatage :0:2 (deux décimales) est une bonne pratique d'affichage.

10.5 Écrire et exécuter la recherche séquentielle (avec témoin)

MÉTHODE Algorithme de recherche séquentielle dans un tableau non trié

Objectif : Trouver la position d'une valeur Val dans Tab[1..N] ou signaler son absence.

Principe : Parcourir le tableau du premier au dernier élément. S'arrêter dès qu'on trouve Val (optimisation) ou à la fin. Étapes (avec drapeau/témoin Trouv Booléen) :

1. Trouv ← Faux; Pos ← 0.
2. i ← 1.
3. Tant que (i ≤ N et Trouv = Faux) faire
 - Si Tab[i] = Val alors Trouv ← Vrai; Pos ← i
 - Sinon i ← i + 1
4. FinTantQue

5. Si Trouv alors Écrire("Trouvé à l'indice ", Pos) sinon Écrire("Non trouvé").

Variante "Pour" (moins efficace car parcourt tout) : Boucle Pour i de 1 à N, test Si Tab[i]=Val, stockage indice, drapeau. La version TantQue s'arrête dès la première occurrence (complexité meilleur cas $O(1)$).

EXERCICE RÉSOLU 13

Recherche d'un matricule dans la base des inscrits à l'Université de Yaoundé I

Énoncé : Un tableau Matricules[1..100] contient les numéros d'inscription (entiers) de 100 étudiants. Écrire l'algorithme de recherche séquentielle d'un matricule saisi par l'utilisateur. Exécuter pas à pas sur l'exemple : Tab = [105, 210, 305, 420, 501], N=5, Val=305.

SOLUTION

Algorithme général :

```

Algorithme RechercheSequentielle
Constante TAILLE = 100
Variables
  Tab : Tableau[1..TAILLE] de Entier
  N, i, Val, Pos : Entier
  Trouv : Booléen
Début
  // ... Saisie de N (<=TAILLE) et remplissage Tab ...
  Écrire("Matricule à rechercher : "); Lire(Val)

  Trouv <- Faux
  Pos <- 0
  i <- 1
  TantQue (i <= N) ET (Trouv = Faux) Faire
    Si Tab[i] = Val Alors
      Trouv <- Vrai
      Pos <- i
    Sinon
      i <- i + 1
    FinSi
  FinTantQue

  Si Trouv Alors
    Écrire("Étudiant trouvé à l'indice ", Pos)
  Sinon
    Écrire("Matricule introuvable dans la base.")
  FinSi
Fin
  
```

Exécution pas à pas (Table d'exécution) pour Val=305 :

Ligne	Instruction	i	Tab[i]	Val	Trouv	Pos
1	Initialisation	1	105	305	F	0
2	Test TantQue ($1 \leq 5$ et F)	1	105	305	F	0
3	Si $105 = 305$? Non	1	105	305	F	0
4	$i \leftarrow 2$	2	210	305	F	0
5	Test TantQue ($2 \leq 5$ et F)	2	210	305	F	0
6	Si $210 = 305$? Non	2	210	305	F	0
7	$i \leftarrow 3$	3	305	305	F	0
8	Test TantQue ($3 \leq 5$ et F)	3	305	305	F	0
9	Si $305 = 305$? OUI	3	305	305	V	3
10	FinTantQue (Trouv=V) → Sortie	-	-	-	V	3

Conclusion : L'algorithme s'arrête à l'indice 3. L'étudiant est trouvé. La table d'exécution prouve la correction de la logique booléenne.

10.6 Écrire et exécuter le tri par insertion

MÉTHODE Algorithme de tri par insertion (ordre croissant)

Objectif : Réorganiser $\text{Tab}[1..N]$ pour que $\text{Tab}[1] \leq \text{Tab}[2] \leq \dots \leq \text{Tab}[N]$.

Invariant de boucle : À l'étape i , le sous-tableau $\text{Tab}[1..i-1]$ est trié. On insère $\text{Tab}[i]$ à sa place dans cette partie triée. Étapes :

1. Pour i de 2 à N faire (on commence à 2, car $\text{Tab}[1]$ est "trié" tout seul).
2. $\text{Cle} \leftarrow \text{Tab}[i]$ (sauvegarde de la valeur à insérer).
3. $j \leftarrow i - 1$ (dernier indice de la zone triée).
4. Tant que ($j \geq 1$ et $\text{Tab}[j] > \text{Cle}$) faire (décalage vers la droite)
 - $\text{Tab}[j+1] \leftarrow \text{Tab}[j]$
 - $j \leftarrow j - 1$
5. FinTantQue
6. $\text{Tab}[j+1] \leftarrow \text{Cle}$ (insertion à la place libérée).
7. FinPour

Point clé : La condition $\text{Tab}[j] > \text{Cle}$ (strict) garantit la stabilité (ordre relatif des égaux conservé). Si $\geq$, le tri devient instable.

EXERCICE RÉSOLU 14**Classement des ventes journalières d'une boutique à Douala Akwa**

Énoncé : On a les ventes (en milliers FCFA) sur 6 jours : Ventes = [45, 12, 78, 12, 33, 60]. Appliquer le tri par insertion. Montrer l'état du tableau après chaque passage de la boucle principale ($i=2..6$).

SOLUTION

Exécution pas à pas (Trace complète) : État initial : [45, 12, 78, 12, 33, 60]

- $i=2$: $Cle=12$. $j=1$. $Tab[1]=45 > 12$? Oui. Décale : $Tab[2]=45$. $j=0$. Fin TantQue. $Tab[1]=12$. → [12, 45, 78, 12, 33, 60]
- $i=3$: $Cle=78$. $j=2$. $Tab[2]=45 > 78$? Non. Fin TantQue immédiat. $Tab[3]=78$. → [12, 45, 78, 12, 33, 60] (inchangé)
- $i=4$: $Cle=12$. $j=3$. $Tab[3]=78 > 12$? Oui. $Tab[4]=78$. $j=2$. $Tab[2]=45 > 12$? Oui. $Tab[3]=45$. $j=1$. $Tab[1]=12 > 12$? Non (strict). Fin. $Tab[2]=12$. → [12, 12, 45, 78, 33, 60] (Stabilité : le 12 d'indice 4 passe APRÈS le 12 d'indice 1).
- $i=5$: $Cle=33$. $j=4$. $78 > 33$? Oui $Tab[5]=78$. $j=3$. $45 > 33$? Oui $Tab[4]=45$. $j=2$. $12 > 33$? Non. $Tab[3]=33$. → [12, 12, 33, 45, 78, 60]
- $i=6$: $Cle=60$. $j=5$. $78 > 60$? Oui $Tab[6]=78$. $j=4$. $45 > 60$? Non. $Tab[5]=60$. → [12, 12, 33, 45, 60, 78]

Tableau final trié : [12, 12, 33, 45, 60, 78]. **Conclusion :** Le tri place les ventes du plus faible au plus fort. Les deux jours à 12 conservent leur ordre relatif (stabilité).

10.7 Résoudre un problème concret : Recherche + Comptage**MÉTHODE****Résolution de problème "Recherche/Comptage dans un tableau"**

Approche type Bac :

1. Analyse : Identifier les données d'entrée (tableau, taille, critère), les sorties (position, nombre, booléen), les traitements (parcours, test).
2. Choix structure : Tableau 1D si données homogènes. Enregistrement si entité complexe (mais alors tableau d'enregistrements).
3. Algorithme : Déclarer → Saisir/Initialiser → Traiter (Boucle + Si) → Afficher résultat.
4. Test de bord : Tableau vide ($N = 0$), Valeur absente, Valeur présente multiples fois (première? toutes?).

Patron "Comptage d'occurrences" : $Nb \leftarrow 0$; Pour i de 1 à N faire Si $Tab[i]=Val$ Alors $Nb \leftarrow Nb+1$ FinSi FinPour

EXERCICE RÉSOLU 15 Analyse des résultats du Baccalauréat – Session Juin

Énoncé : Un tableau `Notes[1..50]` contient les notes de SVT (sur 20) des 50 élèves d'une série D. Écrire un algorithme qui :

1. Calcule et affiche la moyenne de la classe.
2. Compte et affiche le nombre d'élèves ayant une note ≥ 10 (admis) et < 10 (ajournés).
3. Trouve et affiche la note maximale et l'indice (rang) du premier élève l'ayant obtenue.

SOLUTION

Algorithme :

```
Algorithme StatsBacSVT
Constante MAX = 50
Variables
    Notes : Tableau[1..MAX] de Entier
    i, N, Somme, NbAdmis, NbAjournes, MaxNote, IndiceMax : Entier
    Moyenne : Réel
Début
    // Initialisation & Saisie N (supposé <= MAX)
    Écrire("Effectif réel (<=50) : "); Lire(N)
    Somme <- 0
    Pour i de 1 à N faire
        Écrire("Note élève ", i, " : "); Lire(Notes[i])
        Somme <- Somme + Notes[i]
    FinPour

    // 1. Moyenne
    Moyenne <- Somme / N
    Écrire("Moyenne classe : ", Moyenne : 0 : 2)

    // 2. Comptage Admis/Ajournés
    NbAdmis <- 0
    NbAjournes <- 0
    Pour i de 1 à N faire
        Si Notes[i] >= 10 Alors NbAdmis <- NbAdmis + 1
        Sinon NbAjournes <- NbAjournes + 1
    FinSi
    FinPour
    Écrire("Admis : ", NbAdmis, " | Ajournés : ", NbAjournes)

    // 3. Maximum et Premier Indice (Recherche du Max + Position)
    MaxNote <- Notes[1]
    IndiceMax <- 1
    Pour i de 2 à N faire
        Si Notes[i] > MaxNote Alors
            MaxNote <- Notes[i]
            IndiceMax <- i
        FinSi
    FinPour
    Écrire("Meilleure note : ", MaxNote, " (Élève n°", IndiceMax, ")")
Fin
```

Justification :

- Une seule boucle de saisie cumule la somme (efficacité).
- Boucle séparée pour comptage (clarté), ou fusionnable.
- Recherche du Max : initialisation avec le premier élément (`Notes[1]`), boucle à partir de 2. Condition stricte `>` pour garder le premier indice en cas d'égalité.

10.8 Résoudre un problème concret : Tri + Médiane

MÉTHODE Résolution de problème "Tri + Statistique de position (Médiane)"

Approche :

1. Trier le tableau (Insertion ou Bulle/Selection si demandé).
2. Déterminer la médiane selon la parité de N :
 - N impair : Médiane = `Tab[(N+1)/2]` (indice entier).
 - N pair : Médiane = $(\text{Tab}[N/2] + \text{Tab}[N/2 + 1]) / 2$.
3. Attention : Indices 1-based (Bac). En C (0-based) : indices $(N-1)/2$ et $N/2$.

EXERCICE RÉSOLU 16 Calcul de la médiane des salaires d'une PME camerounaise

Énoncé : On dispose des salaires mensuels (en milliers FCFA) de 7 employés : Salaires = [150, 350, 200, 500, 250, 180, 400].

1. Trier ce tableau par insertion (donner l'état final).
2. Calculer la médiane de cette série statistique.
3. Écrire l'algorithme général pour N salaires.

SOLUTION

1. Tri par insertion (résumé des états finaux par passage) : Initial : [150, 350, 200, 500, 250, 180, 400] $i=2$ (Cle=350) : [150, 350, 200, 500, 250, 180, 400] $i=3$ (Cle=200) : [150, 200, 350, 500, 250, 180, 400] $i=4$ (Cle=500) : [150, 200, 350, 500, 250, 180, 400] $i=5$ (Cle=250) : [150, 200, 250, 350, 500, 180, 400] $i=6$ (Cle=180) : [150, 180, 200, 250, 350, 500, 400] $i=7$ (Cle=400) : [150, 180, 200, 250, 350, 400, 500] Tableau trié : [150, 180, 200, 250, 350, 400, 500]

2. Médiane : $N = 7$ (impair). Position centrale = $(7 + 1)/2 = 4^{\text{ème}}$ valeur. `Tab[4]` = 250. Médiane = 250 000 FCFA.

3. Algorithme Général :

Algorithme MedianeSalaires

Variables

Sal : Tableau[1..100] de Entier // Taille max

N, i, j, Cle, Mediane : Entier

MedianeReelle : Réel

Début

// Saisie N et Tableau ...

// TRI PAR INSERTION

Pour i de 2 à N faire

 Cle ← Sal[i]

 j ← i - 1

 TantQue (j >= 1) ET (Sal[j] > Cle) faire

 Sal[j+1] ← Sal[j]

 j ← j - 1

 FinTantQue

 Sal[j+1] ← Cle

FinPour

// CALCUL MÉDIANE

Si N Mod 2 = 1 Alors // N impair (Modulo standard pseudo-code)

 Mediane ← Sal[(N+1)/2]

 Écrire("Médiane = ", Mediane)

Sinon // N pair

 MedianeReelle ← (Sal[N/2] + Sal[N/2 + 1]) / 2.0

 Écrire("Médiane = ", MedianeReelle :0 :2)

FinSi

Fin

Note : L'opérateur Mod (modulo) teste la parité. La division / 2.0 force le calcul réel pour la médiane paire.

10.9 Construire et exploiter une table d'exécution complète

MÉTHODE

Construction rigoureuse d'une table d'exécution (Trace)

Objectif : Prouver le fonctionnement d'un algorithme (boucles, conditions, modifications variables) pour le Bac (souvent 4 à 6 points).

Colonnes obligatoires :

1. Ligne / N° Instruction : Numéro de la ligne exécutée (selon numérotation de l'énoncé).
2. Instruction : Libellé court de l'action (ex : $i \leftarrow 1$, Si Tab[i] > Max).
3. Variables modifiées : Une colonne par variable qui change de valeur dans l'algorithme (indices de boucle, accumulateurs, drapeaux, résultat). Ne pas mettre les constantes ni les variables inchangées.
4. Écran / Sortie : Ce qui s'affiche (Écrire) à cet instant précis.

Règles d'or :

- Une ligne du tableau = Une exécution d'une instruction (pas une itération complète de boucle).
- Initialiser la première ligne avec les valeurs de départ (après saisie/Début).
- Ne pas sauter les lignes de test (Si, TantQue, Pour) : elles font partie du flux.
- Arrêter la trace quand la condition d'arrêt est remplie ou à la fin de l'algorithme.

EXERCICE RÉSOLU 17 Trace complète : Recherche du maximum dans un tableau

Énoncé : Soit l'algorithme suivant (indices 1..N). Construire la table d'exécution complète pour $N=4$, $\text{Tab} = [12, 25, 25, 10]$.

```
1 Algorithme MaxTableau
2 Variables Tab[1..4] : Entier; N, i, Max, Pos : Entier
3 Début
4   N ← 4
5   Tab[1] ← 12; Tab[2] ← 25; Tab[3] ← 25; Tab[4] ← 10 // Initialisation fixe pour trace
6   Max ← Tab[1] // L1
7   Pos ← 1 // L2
8   Pour i de 2 à N faire // L3
9     Si Tab[i] > Max alors // L4
10      Max ← Tab[i] // L5
11      Pos ← i // L6
12   FinSi // L7
13   FinPour // L8
14   Écrire(Max, Pos) // L9
15 Fin
```

SOLUTION

Table d'exécution :

Ligne	Instruction	i	Max	Pos	Tab[i]	Écran
L1	Max ← Tab[1]	-	12	-	12	
L2	Pos ← 1	-	12	1	-	
L3	Pour i=2 (init)	2	12	1	25	
L4	Si Tab[2] > Max (25>12)? V	2	12	1	25	
L5	Max ← Tab[2]	2	25	1	25	
L6	Pos ← 2	2	25	2	25	
L7	FinSi	2	25	2	-	
L3	Pour i=3 (incr)	3	25	2	25	
L4	Si Tab[3] > Max (25>25)? F	3	25	2	25	
L7	FinSi	3	25	2	-	
L3	Pour i=4 (incr)	4	25	2	10	
L4	Si Tab[4] > Max (10>25)? F	4	25	2	10	
L7	FinSi	4	25	2	-	
L3	FinPour (i=5 > N)	-	25	2	-	
L9	Écrire(Max, Pos)	-	25	2	-	25 2

Analyse : L'algorithme retourne $Max=25$, $Pos=2$. La condition stricte $>$ conserve la position de la première occurrence du maximum (indice 2), ignorant la seconde (indice 3). La trace valide ce comportement.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Indices de tableau : Confondre base 1 (Pseudo-code Bac) et base 0 (Langage C). Règle : En pseudo-code officiel Cameroun, les tableaux sont indicés de 1 à N. Écrire Pour i de 0 à N-1 en pseudo-code est une erreur majeure. En C, c'est l'inverse.
2. Drapeau (Témoin) non initialisé : Oublier $Trouv \leftarrow \text{Faux}$ avant la boucle de recherche. La variable contient une valeur aléatoire → logique imprévisible.
3. Condition de boucle TantQue inversée : Écrire TantQue (Trouv) Faire au lieu de TantQue (Non Trouv) Faire ou TantQue ($i \leq N$ ET Non Trouv). Cela fait soit 0 tour, soit boucle infinie.
4. Décalage dans Tri Insertion : Écrire $Tab[j] \leftarrow Tab[j+1]$ (vers la gauche) au lieu de $Tab[j+1] \leftarrow Tab[j]$ (vers la droite pour faire de la place). Perte de données garantie.
5. Table d'exécution incomplète : Ne pas tracer les lignes de test (Si, TantQue, FinPour) ou ne pas mettre la valeur de $Tab[i]$ dans une colonne dédiée quand elle est utilisée dans un test. Le correcteur ne peut pas suivre le raisonnement → 0 point sur la trace.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 5 QCM : structure d'un algorithme et tableaux

Parmi les propositions suivantes, une seule est exacte. Laquelle ?

1. Dans un algorithme, la déclaration `Tableau T[10] : Entier` réserve 10 cases d'indices 1 à 10.
 2. L'instruction `T[0] ← 5` affecte la valeur 5 à la première case d'un tableau déclaré `T[10]` en indexation base 0.
 3. La structure `Pour i ← 1 à 10 Faire` exécute le corps de la boucle 9 fois.
 4. Un enregistrement (structure) ne peut contenir que des champs de même type.
- Justifie ton choix en expliquant pourquoi les trois autres sont fausses.

EXERCICE 6 Vrai ou Faux justifié : recherche séquentielle

Détermine si les affirmations suivantes sont vraies ou fausses. Justifie chaque réponse.

1. L'algorithme de recherche séquentielle nécessite que le tableau soit trié au préalable.
2. La variable « drapeau » (ou témoin) est initialisée à `Vrai` avant d'entrer dans la boucle de recherche.
3. Si la valeur cherchée est présente plusieurs fois, l'algorithme standard avec drapeau s'arrête à la première occurrence trouvée.
4. La condition de continuation de la boucle `TantQue` est généralement : $(i \leq N)$ ET $(\text{drapeau} = \text{Faux})$.

EXERCICE 7 Définitions et vocabulaire

Donne une définition précise et rigoureuse pour chacun des termes suivants :

1. Structure de données enregistrement (ou type structuré).
2. Parcours d'un tableau.
3. Table d'exécution (ou trace d'exécution).
4. Complexité en temps d'un algorithme de tri par insertion dans le pire des cas.

EXERCICE 8 Calculs directs sur un tri par insertion

Soit le tableau T d'entiers suivant (indexation base 1) : $T = [14, 33, 27, 10, 35, 19, 42, 44]$. On exécute l'algorithme de tri par insertion standard (boucle externe pour i de 2 à N).

1. Combien de comparaisons entre éléments du tableau sont effectuées au total pour trier complètement ce tableau ? (Donne le nombre exact, pas une approximation big-O).
2. Quel est l'état du tableau après l'itération $i = 3$ (c'est-à-dire après l'insertion de l'élément d'indice 3 dans la partie triée) ?

11.2 Je m'entraîne**EXERCICE 9** Notes d'élèves : maximum et compteur de moyenne

On souhaite traiter les notes de 20 élèves d'une classe de Terminale C au Lycée de Nkol-bisson. Les notes sont des entiers compris entre 0 et 20. Écrire un algorithme complet qui :

1. Déclare un tableau `Notes[20]` et les variables nécessaires.
2. Remplit le tableau en demandant la note de chaque élève (saisie utilisateur).
3. Détermine et affiche la note maximale obtenue.
4. Compte et affiche le nombre d'élèves ayant la moyenne (note ≥ 10).

Utilise une structure `Pour` pour le remplissage et une structure `Pour` (ou `TantQue`) pour le traitement. Soigne l'affichage des messages.

EXERCICE 10 Gestion de stock : enregistrement Produit

Une boutique informatique à Douala (quartier Akwa) gère un stock de 15 produits. Chaque produit est caractérisé par :

- `son nom` (chaîne de caractères),
- `son prixUnitaire` (réel, en FCFA),
- `sa quantiteStock` (entier).

1. Écrire la déclaration du type `Produit` (enregistrement) et la déclaration d'un tableau `Stock[15]` de ce type.
2. Écrire un algorithme qui parcourt le tableau `Stock` pour afficher le nom et le prix de tous les produits dont la quantité en stock est strictement inférieure à 5 (seuil d'alerte de

réapprovisionnement).

3. Modifier l'algorithme pour qu'il calcule et affiche la valeur totale du stock (somme des $\text{prixUnitaire} \times \text{quantiteStock}$ pour tous les produits).

EXERCICE 11 Recherche séquentielle : algorithme complet et trace

On considère un tableau $T[10]$ d'entiers déjà rempli : [12, 5, 28, 17, 5, 33, 9, 41, 22, 5]. On cherche la valeur $V = 5$.

1. Écrire l'algorithme complet de la recherche séquentielle avec drapeau qui affiche « Trouvé à l'indice i » pour la première occurrence, ou « Non trouvé » si la valeur est absente.
2. Réaliser la table d'exécution complète (colonnes : i , $T[i]$, Drapeau, Écran) pour cette recherche.
3. Que se passerait-il si on oubliait d'initialiser le drapeau à Faux avant la boucle? Explique.

EXERCICE 12 Tri par insertion : exécution pas à pas

Soit le tableau $T[6]$ (indices 1 à 6) initial : [25, 12, 30, 7, 18, 12]. On applique l'algorithme de tri par insertion standard (tri croissant).

1. Donner l'état du tableau après chaque itération de la boucle externe (c'est-à-dire pour $i = 2, i = 3, i = 4, i = 5, i = 6$). Présente le résultat sous forme de tableau.
2. Combien de décalages (affectations $T[j+1] \leftarrow T[j]$) ont été effectués au total pendant tout le tri?
3. Le tri par insertion est-il stable? Justifie ta réponse en t'appuyant sur l'exemple ci-dessus (remarque les deux valeurs 12).

11.3 Je me prépare au Bac

EXERCICE 13 Sujet type Bac : Correction d'algorithme et table d'exécution — 10 pts

On donne ci-dessous un algorithme incorrect de recherche séquentielle dans un tableau $T[1..N]$ d'entiers. La valeur cherchée est Val . Trois erreurs volontaires y ont été glissées.

Algorithme Recherche_Erronee
Variables

```

T : Tableau [1..N] d'Entier
N, i, Val : Entier
Trouve : Booleen
Début
  // Remplissage du tableau omis
  Ecrire("Valeur à chercher : ")
  Lire(Val)
  i <- 1
  TantQue (i <= N) ET (Trouve = Vrai) Faire
    Si T[i] = Val Alors
      Trouve <- Vrai
    Sinon
      i <- i + 1
    FinSi
  FinTantQue
  Si Trouve = Vrai Alors
    Ecrire("Trouvé à l'indice ", i)
  Sinon
    Ecrire("Non trouvé")
  FinSi
Fin

```

1. Identifie et explique clairement les trois erreurs présentes dans cet algorithme. (3 pts)
2. Propose la version corrigée complète de l'algorithme (réécris-le intégralement). (3 pts)
3. On exécute l'algorithme corrigé avec $N = 5$, $T = [10, 20, 30, 40, 50]$ et $Val = 30$. Réalise la table d'exécution complète (colonnes : i , $T[i]$, Trouve, Condition TantQue, Écran). (4 pts)

EXERCICE 14 Sujet type Bac : Tri par insertion et analyse — 10 pts

Une application de gestion de bibliothèque scolaire (Lycée de Bafoussam) doit trier les codes ISBN (entiers sur 5 chiffres) de N ouvrages par ordre croissant. On utilise le tri par insertion. On donne le tableau initial $T[1..5] = [25, 12, 30, 7, 18]$.

1. Écrire l'algorithme complet du tri par insertion croissant sur un tableau $T[1..N]$. Utilise des noms de variables explicites (i , j , cle , etc.). (3 pts)
2. Effectue la trace pas à pas (table d'exécution) de cet algorithme sur le tableau donné. Présente l'état du tableau après chaque insertion (c'est-à-dire à la fin de chaque tour de la boucle pour i). (4 pts)
3. Quelle est la complexité en temps (notation grand O) de ce tri dans le pire des cas ? Dans quel cas de figure se produit ce pire cas ? (1,5 pt)
4. L'algorithme de tri par insertion est-il en place ? Justifie. (1,5 pt)

EXERCICE 15 **Sujet type Bac : Problème complet — Recherche, tri et enregistrement — 10 pts**

La société CAMTEL souhaite automatiser la gestion des abonnés fibre optique dans l'agence de Yaoundé-Mvog-Ada. On modélise un abonné par un enregistrement :

Type Abonne = Enregistrement

NumContrat : Chaîne // ex: "CM-YDE-2024-00123"

NomComplet : Chaîne

DebitMbit : Entier // débit souscrit en Mbit/s (ex: 10, 50, 100)

Mensualite : Reel // en FCFA

Fin Enregistrement

On déclare un tableau `Clients[1..50]` de type `Abonne`. Le nombre réel d'abonnés actuels est stocké dans `NbClients` (≤ 50).

1. Déclaration et saisie : Écrire l'algorithme qui permet de saisir les informations des `NbClients` abonnés (boucle de remplissage du tableau). (2 pts)
2. Recherche par numéro de contrat : Écrire un algorithme (ou fonction) `RechercheContrat(Clients, NbClients, NumRecherche)` qui renvoie l'indice de l'abonné dont le `NumContrat` correspond à `NumRecherche`, ou 0 si absent. Utilise une recherche séquentielle avec drapeau. (3 pts)
3. Tri par débit décroissant : La direction veut afficher la liste des clients triée par `DebitMbit` décroissant (les gros débits d'abord). Adapter l'algorithme de tri par insertion pour trier le tableau `Clients` selon ce critère. (3 pts)
4. Statistique : Écrire l'algorithme qui calcule et affiche le chiffre d'affaires mensuel total (somme des `Mensualite`) et le nombre d'abonnés ayant un débit ≥ 100 Mbit/s. (2 pts)

12 CORRIGÉS DES EXERCICES

13 EXERCICES CORRIGÉS — LEÇON 07 : ÉCRITURE ET EXÉCUTION D'ALGORITHMES INTÉGRANT UNE STRUCTURE DE DONNÉES

CORRIGÉ Exercice 1 — QCM : structure d'un algorithme et tableaux

Réponse exacte : la proposition 2.

- Proposition 1 — Fausse. La déclaration `Tableau T[10] : Entier` réserve bien 10 cases, mais en indexation **base 0** (convention du langage C), les indices vont de 0 à 9, et non de 1 à 10. La proposition serait vraie uniquement en indexation base 1 (pseudo-code classique au Bac).
- Proposition 2 — Vraie. En indexation base 0, la première case du tableau porte l'indice 0. L'instruction `T[0] ← 5` affecte donc correctement la valeur 5 à la première case. C'est la seule proposition exacte dans ce contexte.
- Proposition 3 — Fausse. La boucle `Pour i ← 1 à 10 Faire` exécute son corps pour $i = 1, 2, \dots, 10$, soit $10 - 1 + 1 = 10$ itérations, et non 9.
- Proposition 4 — Fausse. C'est précisément l'intérêt de l'**enregistrement** : il regroupe des champs de types différents (par exemple un nom de type chaîne, un prix de type réel, une quantité de type entier). La structure qui impose l'homogénéité des types est le tableau, pas l'enregistrement.

Point de vigilance : au Bac, lis toujours attentivement la convention d'indexation (base 0 ou base 1) indiquée dans l'énoncé. En pseudo-code algorithmique (épreuve écrite), l'indexation base 1 ($1..N$) est la norme ; en langage C (épreuve pratique), c'est systématiquement la base 0 ($0..N-1$).

CORRIGÉ Exercice 2 — Vrai ou Faux : recherche séquentielle

1. Faux. La recherche **séquentielle** parcourt le tableau élément par élément dans l'ordre : elle ne suppose aucun ordre préalable. C'est la recherche dichotomique (hors programme) qui exige un tableau trié.
2. Faux. Le drapeau est initialisé à `Faux` avant la boucle : il signifie « pas encore trouvé ». Il ne passe à `Vrai` que lorsqu'on rencontre la valeur cherchée. L'initialiser à `Vrai` empêcherait la boucle de s'exécuter (voir la condition de la question 4).

3. Vrai. Dès que $T[i] = Val$, le drapeau passe à Vrai, la condition (`drapeau = Faux`) devient fausse et la boucle s'arrête immédiatement : on obtient donc la première occurrence.
4. Vrai. On continue tant que deux conditions sont réunies : on n'a pas dépassé la fin du tableau ($i \leq N$) et on n'a pas encore trouvé (`drapeau = Faux`). Les deux conditions sont indispensables : la première évite le débordement d'indice, la seconde permet l'arrêt anticipé.

CORRIGÉ Exercice 3 — Définitions et vocabulaire

1. Enregistrement (type structuré) : structure de données finie qui regroupe sous un même nom plusieurs variables appelées **champs**, pouvant être de types différents. Chaque champ est accessible individuellement par la notation pointée : `Variable.NomDuChamp`.
2. Parcours d'un tableau : action d'accéder successivement, au moyen d'une boucle (généralement `Pour`), à chacun des éléments du tableau, du premier au dernier indice, afin d'effectuer un traitement (lecture, affichage, recherche, somme...).
3. Table d'exécution (trace) : tableau à colonnes (numéro de ligne ou instruction, valeurs des variables, affichages à l'écran) qui décrit l'évolution de l'état de la mémoire et des sorties au fil de l'exécution pas à pas d'un algorithme. C'est l'outil de vérification et de validation d'une solution algorithmique.
4. Complexité en temps du tri par insertion (pire des cas) : dans le pire des cas (tableau trié en ordre décroissant), chaque élément $T[i]$ est comparé à tous les éléments de la partie déjà triée, soit $1 + 2 + \dots + (N - 1) = \frac{N(N - 1)}{2}$ comparaisons. La complexité en temps est donc **quadratique** : $O(N^2)$.

CORRIGÉ Exercice 4 — Calculs directs sur un tri par insertion

Tableau initial (base 1) : $T = [14, 33, 27, 10, 35, 19, 42, 44]$, $N = 8$.

1. Nombre total de comparaisons. On compte, pour chaque valeur de i , les comparaisons $T[j] > cle$ effectuées (la boucle `TantQue` s'arrête dès qu'une comparaison est fausse ou que $j = 0$) :

i	cle	Comparaisons	Détail
2	33	1	33 < 14? Non → arrêt
3	27	2	27 < 33? Oui; 27 < 14? Non
4	10	3	10 < 33, 10 < 27, 10 < 14 : Oui; j atteint 0
5	35	1	35 < 33? Non
6	19	4	19 < 35, 19 < 33, 19 < 27 : Oui; 19 < 14? Non
7	42	1	42 < 35? Non
8	44	1	44 < 42? Non

Total : 1 + 2 + 3 + 1 + 4 + 1 + 1 = 13 comparaisons.

2. État après l'itération $i = 3$. On insère $cle = 27$ dans la partie triée [14, 33] : 33 est décalé d'une case, 27 prend sa place :

$T = [14, 27, 33, 10, 35, 19, 42, 44]$

CORRIGÉ

Exercice 5 — Notes d'élèves : maximum et compteur de moyenne

Algorithme Notes_Classe

Constantes

NB = 20

Variables

Notes : Tableau [1..NB] d'Entier

i, Max, NbMoyenne : Entier

Début

// 1. Remplissage du tableau

Pour i ← 1 à NB Faire

 Ecrire("Entrez la note de l'élève n° ", i, " : ")

 Lire(Notes[i])

FinPour

// 2. Initialisation du traitement

Max ← Notes[1]

NbMoyenne ← 0

// 3. Parcours : maximum et compteur de moyenne

Pour i ← 1 à NB Faire

 Si Notes[i] > Max Alors

 Max ← Notes[i]

 FinSi

 Si Notes[i] ≥ 10 Alors

 NbMoyenne ← NbMoyenne + 1

 FinSi

FinPour

// 4. Affichage des résultats

Ecrire("La note maximale de la classe est : ", Max)

Ecrire("Nombre d'élèves ayant la moyenne : ", NbMoyenne)

Fin**Remarques :**

- Max est initialisé à Notes[1] (et non à 0) : c'est la méthode correcte, qui fonctionne même si toutes les notes étaient négatives.
- Un seul parcours suffit pour les deux traitements : c'est plus efficace que deux boucles séparées.
- La condition de la moyenne est ≥ 10 (note ≥ 10), attention à ne pas écrire > 10 .

CORRIGÉ Exercice 6 — Gestion de stock : enregistrement Produit**1. Déclarations.**

```
Type Produit = Enregistrement
    nom : Chaîne
    prixUnitaire : Réel
    quantiteStock : Entier
Fin Enregistrement

Variables
    Stock : Tableau [1..15] de Produit
```

2. Alerte de réapprovisionnement (quantité < 5).

```
Algorithme Alerte_Stock
Variables
    Stock : Tableau [1..15] de Produit
    i : Entier
Début
    // Remplissage du tableau omis
    Ecrire("Produits à réapprovisionner :")
    Pour i <- 1 à 15 Faire
        Si Stock[i].quantiteStock < 5 Alors
            Ecrire(Stock[i].nom, " - Prix : ",
                Stock[i].prixUnitaire, " FCFA")
        FinSi
    FinPour
Fin
```

3. Valeur totale du stock.

```
Algorithme Valeur_Stock
Variables
    Stock : Tableau [1..15] de Produit
    i : Entier
    ValeurTotale : Réel
Début
    // Remplissage du tableau omis
    ValeurTotale <- 0
    Pour i <- 1 à 15 Faire
```

```

    ValeurTotale <- ValeurTotale
    + Stock[i].prixUnitaire * Stock[i].quantiteStock
  FinPour
  Ecrire("Valeur totale du stock : ", ValeurTotale, " FCFA")
Fin

```

Points de vigilance : l'accès à un champ se fait par la notation pointée `Stock[i].quantiteStock` (indice puis champ); l'accumulateur `ValeurTotale` doit être initialisé à 0 avant la boucle ; la condition du seuil est stricte (`< 5`).

CORRIGÉ Exercice 7 — Recherche séquentielle : algorithme complet et trace

1. Algorithme avec drapeau.

```

Algorithme Recherche_Sequentielle
Variables
  T : Tableau [1..10] d'Entier
  i, V : Entier
  Drapeau : Booléen
Début
  // Remplissage du tableau omis
  Ecrire("Valeur à chercher : ")
  Lire(V)
  i <- 1
  Drapeau <- Faux
  TantQue (i <= 10) ET (Drapeau = Faux) Faire
    Si T[i] = V Alors
      Drapeau <- Vrai
    Sinon
      i <- i + 1
    FinSi
  FinTantQue
  Si Drapeau = Vrai Alors
    Ecrire("Trouvé à l'indice ", i)
  Sinon
    Ecrire("Non trouvé")
  FinSi
Fin

```

2. Table d'exécution pour $T = [12, 5, 28, 17, 5, 33, 9, 41, 22, 5]$ et $V = 5$:

i	T[i]	Drapeau	Écran
1	12	Faux	
2	5	Faux → Vrai	
Sortie de boucle (Drapeau = Vrai)			Trouvé à l'indice 2

Détail : à $i = 1$, $T[1] = 12 \neq 5$, donc $i \leftarrow 2$. À $i = 2$, $T[2] = 5 = V$, le drapeau passe à Vrai ; la condition (Drapeau = Faux) devient fausse, la boucle s'arrête. On affiche « Trouvé à l'indice 2 » : c'est bien la première occurrence (les autres 5, aux indices 5 et 10, ne sont pas examinés).

3. Oubli de l'initialisation du drapeau. La variable `Drapeau` contiendrait une valeur indéterminée (résiduelle en mémoire). Si cette valeur est `Vrai`, la condition du `TantQue` est fausse dès le premier test : la boucle ne s'exécute jamais et l'algorithme affiche « Non trouvé » même si la valeur est présente. Le comportement devient imprévisible : c'est un bug classique. Toute variable utilisée dans une condition doit être initialisée avant la boucle.

CORRIGÉ Exercice 8 — Tri par insertion : exécution pas à pas

Tableau initial : $T = [25, 12, 30, 7, 18, 12]$.

1. États successifs du tableau.

i	cle	État du tableau après insertion	Décalages
2	12	[12, 25, 30, 7, 18, 12]	1
3	30	[12, 25, 30, 7, 18, 12]	0
4	7	[7, 12, 25, 30, 18, 12]	3
5	18	[7, 12, 18, 25, 30, 12]	2
6	12	[7, 12, 12, 18, 25, 30]	3

Détail des décalages : $i = 2$: 25 recule (1) ; $i = 3$: 30 est déjà à sa place (0) ; $i = 4$: 30, 25, 12 reculent (3) ; $i = 5$: 30 et 25 reculent (2) ; $i = 6$: 30, 25, 18 reculent, mais le premier 12 arrête le décalage car $12 > 12$ est faux (3).

2. Nombre total de décalages : $1 + 0 + 3 + 2 + 3 = 9$ décalages.

3. Stabilité. Oui, le tri par insertion est stable : deux éléments égaux conservent leur ordre relatif initial. Ici, le 12 d'origine indice 2 se retrouve à l'indice 2 du tableau trié, et le 12 d'origine indice 6 à l'indice 3 : le premier 12 reste avant le second. Cela provient de la condition stricte de décalage $T[j] > cle$: un élément égal à la clé n'est jamais décalé, donc la clé est insérée après les éléments qui lui sont égaux.

CORRIGÉ Exercice 9 — Sujet type Bac : Correction d'algorithme et table d'exécution

1. Les trois erreurs (3 pts — 1 pt par erreur identifiée et expliquée).

- Erreur 1 : le drapeau `Trouve` n'est jamais initialisé avant la boucle. Il faut écrire `Trouve ← Faux` avant le `TantQue`.
- Erreur 2 : la condition de continuation est inversée : $(Trouve = Vrai)$ doit être $(Trouve = Faux)$. On continue tant qu'on n'a pas trouvé.
- Erreur 3 : l'affectation dans le `Si` est inversée : quand $T[i] = Val$, il faut écrire `Trouve ← Vrai` (et non `Faux`), sinon le drapeau ne signale jamais la découverte.

(Conséquence de ces erreurs : la boucle ne s'exécute jamais ou ne s'arrête jamais cor-

rectement, et le message final est faux.)

2. Version corrigée (3 pts).

```

Algorithme Recherche_Corrigee
Variables
  T : Tableau [1..N] d'Entier
  N, i, Val : Entier
  Trouve : Booléen
Début
  // Remplissage du tableau omis
  Ecrire("Valeur à chercher : ")
  Lire(Val)
  i <- 1
  Trouve <- Faux
  TantQue (i <= N) ET (Trouve = Faux) Faire
    Si T[i] = Val Alors
      Trouve <- Vrai
    Sinon
      i <- i + 1
    FinSi
  FinTantQue
  Si Trouve = Vrai Alors
    Ecrire("Trouvé à l'indice ", i)
  Sinon
    Ecrire("Non trouvé")
  FinSi
Fin
  
```

3. Table d'exécution (4 pts) avec $N = 5$, $T = [10, 20, 30, 40, 50]$, $Val = 30$:

i	T[i]	Trouve	Condition TantQue	Écran
1	10	Faux	$(1 \leq 5)$ ET Vrai = Vrai	
2	20	Faux	$(2 \leq 5)$ ET Vrai = Vrai	
3	30	Faux → Vrai	$(3 \leq 5)$ ET Faux = Faux	
Sortie de boucle, test final : Trouve = Vrai				Trouvé à l'indice 3

Points de vigilance (barème) : l'examineur attend que chaque erreur soit non seulement repérée mais expliquée (conséquence sur l'exécution) ; dans la trace, la colonne « Condition » doit montrer explicitement l'évaluation des deux sous-conditions ; ne pas incrémenter i quand on a trouvé (l'indice affiché doit être celui de la découverte).

CORRIGÉ

Exercice 10 — Sujet type Bac : Tri par insertion et analyse

1. Algorithme du tri par insertion (3 pts).

```

Algorithme Tri_Insertion
Variables
  T : Tableau [1..N] d'Entier
  N, i, j, cle : Entier
  
```

```

Début
  // Remplissage du tableau omis
  Pour i <- 2 à N Faire
    cle <- T[i]
    j <- i - 1
    TantQue (j >= 1) ET (T[j] > cle) Faire
      T[j+1] <- T[j]  // décalage vers la droite
      j <- j - 1
    FinTantQue
    T[j+1] <- cle      // insertion de la clé
  FinPour
Fin

```

2. Trace pas à pas (4 pts) sur $T = [25, 12, 30, 7, 18]$:

i	cle	Tableau après insertion	Décalages effectués
2	12	[12, 25, 30, 7, 18]	25
3	30	[12, 25, 30, 7, 18]	aucun
4	7	[7, 12, 25, 30, 18]	30, 25, 12
5	18	[7, 12, 18, 25, 30]	30, 25

Le tableau final trié est $[7, 12, 18, 25, 30]$.

3. Complexité (1,5 pt). Dans le pire des cas, la complexité en temps est $O(N^2)$ (quadratique), car le nombre de comparaisons vaut $1 + 2 + \dots + (N - 1) = \frac{N(N - 1)}{2}$. Ce pire cas se produit lorsque le tableau est initialement trié en ordre décroissant : chaque clé doit être comparée à tous les éléments de la partie déjà triée.

4. Tri « en place » (1,5 pt). Oui : le tri par insertion est **en place** car il ne nécessite aucun tableau auxiliaire ; les échanges se font directement dans le tableau T , avec seulement quelques variables supplémentaires (cle, i, j) dont le nombre ne dépend pas de N (mémoire auxiliaire en $O(1)$).

Points de vigilance (barème) : la boucle externe doit démarrer à $i = 2$; la sauvegarde $cle \leftarrow T[i]$ avant les décalages est obligatoire (sinon la valeur est écrasée) ; la condition du **TantQue** doit comporter $j \geq 1$ en premier pour éviter l'accès à $T[0]$; pour la question 4, citer la mémoire auxiliaire constante est la justification attendue.

CORRIGÉ

Exercice 11 — Sujet type Bac : Problème complet CAMTEL

1. Déclaration et saisie (2 pts).

```

Type Abonne = Enregistrement
  NumContrat : Chaîne
  NomComplet : Chaîne
  DebitMbit : Entier
  Mensualite : Réel
Fin Enregistrement

```

Algorithme Saisie_Abonnes

Variables

Clients : Tableau [1..50] d'Abonne

NbClients, i : Entier

Début

Ecrire("Nombre d'abonnés (<= 50) : ")

Lire(NbClients)

Pour i <- 1 à NbClients Faire

Ecrire(" --- Abonné n° ", i, " ---")

Ecrire("Numéro de contrat : ") ; Lire(Clients[i].NumContrat)

Ecrire("Nom complet : ") ; Lire(Clients[i].NomComplet)

Ecrire("Débit (Mbit/s) : ") ; Lire(Clients[i].DebitMbit)

Ecrire("Mensualité (FCFA) : ") ; Lire(Clients[i].Mensualite)

FinPour

Fin

2. Recherche par numéro de contrat (3 pts).

Fonction RechercheContrat(Clients : Tableau d'Abonne,
NbClients : Entier, NumRecherche : Chaîne) : Entier

Variables

i : Entier

Trouve : Booléen

Début

i <- 1

Trouve <- Faux

TantQue (i <= NbClients) ET (Trouve = Faux) Faire

Si Clients[i].NumContrat = NumRecherche Alors

Trouve <- Vrai

Sinon

i <- i + 1

FinSi

FinTantQue

Si Trouve = Vrai Alors

Retourner i

Sinon

Retourner 0

FinSi

Fin

3. Tri par débit décroissant (3 pts). Il suffit d'inverser le sens de la comparaison : on décrocale tant que le débit rencontré est inférieur à celui de la clé. Attention : la clé est ici un enregistrement complet.

Algorithme Tri_Debit_Decroissant

Variables

Clients : Tableau [1..50] d'Abonne

NbClients, i, j : Entier

cle : Abonne

Début

Pour i <- 2 à NbClients Faire

cle <- Clients[i]

j <- i - 1

TantQue (j >= 1) ET (Clients[j].DebitMbit < cle.DebitMbit) Faire

Clients[j+1] <- Clients[j]

```
        j <- j - 1
    FinTantQue
    Clients[j+1] <- cle
FinPour
Fin
```

4. Statistiques (2 pts).

```
Algorithme Statistiques
Variables
    Clients : Tableau [1..50] d'Abonne
    NbClients, i, NbGrosDebit : Entier
    ChiffreAffaires : Réel
Début
    ChiffreAffaires <- 0
    NbGrosDebit <- 0
    Pour i <- 1 à NbClients Faire
        ChiffreAffaires <- ChiffreAffaires + Clients[i].Mensualite
        Si Clients[i].DebitMbit >= 100 Alors
            NbGrosDebit <- NbGrosDebit + 1
        FinSi
    FinPour
    Ecrire("Chiffre d'affaires mensuel : ", ChiffreAffaires, " FCFA")
    Ecrire("Abonnés avec débit >= 100 Mbit/s : ", NbGrosDebit)
Fin
```

Points de vigilance (barème) : question 2, la fonction doit retourner 0 en cas d'absence (et non un booléen) et ne pas incrémenter *i* lors de la découverte ; question 3, la clé *cle* doit être de type *Abonne* (on déplace des enregistrements entiers, pas seulement les débits, sinon les données seraient mélangées) et l'inversion du sens de tri se fait sur la comparaison *< cle.DebitMbit* ; question 4, les accumulateurs doivent être initialisés avant la boucle et la condition est *>= 100* (au sens large).

14 FICHE BILAN



Carte mentale de la leçon 07 : Algorithmique et structures de données

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Algorithme** : Suite finie et ordonnée d'instructions permettant de résoudre un problème.
- **Variable** : Zone mémoire nommée dont la valeur peut changer pendant l'exécution (Variable Nom : Type).
- **Constante** : Valeur figée dès la déclaration (Constante Nom = Valeur).
- **Tableau** : Structure indexée, collection ordonnée d'éléments de même type, acces-

sibles par un indice (`Tab[1..N] : Type`).

- **Enregistrement** : Structure finie hétérogène regroupant des champs nommés et typés (`Type Nom = Enregistrement ... Fin`).
- **Recherche séquentielle** : Parcours linéaire d'un tableau jusqu'à trouver la valeur cherchée (ou épuiser le tableau). Utilise un **booléen témoin** (`Trouve`).
- **Tri par insertion** : Construction progressive de la zone triée : pour chaque nouvel élément (clé), décalage des éléments plus grands vers la droite.
- **Table d'exécution** : Outil de traçage pas à pas (colonnes : N° ligne, Instruction, Variables modifiées, Écran).

Méthodes express

Action	Étapes clés / Syntaxe
Déclarer & parcourir un tableau Pour i de 1 à N faire Lire(Tab[i]) / Afficher(Tab[i]) FinPour	Tab[1..N] : Entier
Déclarer & utiliser un enregistrement Nom : Chaine Note : Reel FinEnregistrement Variable E : Eleve E.Nom ← "Kamga"	Type Eleve = Enregistrement
Recherche séquentielle (avec drapeau) Tant que (i ≤ N) ET (Non Trouve) faire Si Tab[i] = Val alors Trouve ← Vrai Sinon i ← i + 1 FinTantQue	Trouve ← Faux ; i ← 1
Tri par insertion Cle ← Tab[i] ; j ← i-1 Tant que (j ≥ 1) ET (Tab[j] > Cle) faire Tab[j+1] ← Tab[j] ; j ← j-1 FinTantQue Tab[j+1] ← Cle FinPour	Pour i de 2 à N faire
Construire une table d'exécution	1. Lister les numéros de ligne (ordre d'exécution réel). 2. Noter l'instruction exécutée. 3. Mettre à jour uniquement les variables modifiées. 4. Noter les affichages (Écran).

Complexités à connaître (pire cas)

Algorithme	Complexité temporelle	Complexité spatiale
Recherche séquentielle	$O(n)$	$O(1)$
Tri par insertion	$O(n^2)$	$O(1)$ (tri sur place)

À RETENIR

Checklist avant le Bac

- ☐ Je sais écrire l'en-tête complet d'un algorithme (Algorithme Nom / Variables / Constantes / Début / ... / Fin).
- ☐ Je distingue **Variable** (modifiable) et **Constante** (figée) et je les déclare correctement.
- ☐ Je déclare un tableau : `Tab[1..N] : Type` et je le parcours avec `Pour i de 1 à N`.
- ☐ Je déclare un type **Enregistrement** et j'accède à un champ par `Variable.Champ`.
- ☐ J'écris l'algorithme de **recherche séquentielle** avec un drapeau `Trouve` (booléen) et une boucle `Tant que`.
- ☐ J'écris l'algorithme de **tri par insertion** : boucle externe `Pour i de 2 à N`, clé, décalage dans `Tant que`, insertion finale.
- ☐ Je construis une **table d'exécution** rigoureuse (colonnes : Ligne, Instruction, Variables, Écran) sans sauter d'étape.
- ☐ Je gère les cas limites : tableau vide ($N = 0$), tableau à 1 élément, valeur non trouvée, tableau déjà trié.
- ☐ Je connais les complexités : Recherche $O(n)$, Tri insertion $O(n^2)$ (pire cas), $O(n)$ (meilleur cas déjà trié).
- ☐ Je traduis sans erreur l'algorithme en langage C (indices $0..N-1$, `scanf/printf`, types `int/float/struct`).
- ☐ Je justifie la correction de mes boucles (initialisation, condition d'arrêt, progression, invariants simples).



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+